Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

Architecture of Sintered-Regolith Lunar Polar Microhabitats for Biofilm Research

Miguel Angel Vargas Cruz

miguelangel@grupoalianzaempresarial.com

Grupo Alianza Empresarial

Research Article

Keywords:

Posted Date: December 12th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-8290162/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

Architecture of Sintered-Regolith Lunar Polar Microhabitats for Biofilm Research

Miguel Angel Vargas Cruz

Grupo Alianza Empresarial, Mexico City, Mexico. miguelangel@grupoalianzaempresarial.com

November 30, 2025

Abstract

This paper presents a parametric design framework for sintered-regolith microhabitats at the lunar poles and couples it to a low-order model of biofilm attachment, growth, and radiative damage on internal liner materials. The approach integrates three classes of open datasets: (i) polar illumination geometry derived from LOLA digital elevation models and horizon-based solar-track analysis; (ii) regolith thermophysical properties from Diviner radiometry and in-situ resource utilization (ISRU) studies of solar and microwave sintering; and (iii) International Space Station (ISS) microbiology data, including environmental metagenomes, spaceflight biofilm experiments, and the OSD-554 Space Biofilms study on Pseudomonas aeruginosa.

On the physical side, the framework parameterizes spherical-cap shells in terms of radius, rise, and thickness, estimates fractional illumination and UV dose from site-specific horizons, and solves one-dimensional multilayer conduction problems under polar boundary conditions to evaluate internal temperature ranges and sintering energy requirements. On the biological side, it employs a surface-attachment Monod model with a UV damage term, uses ISS-derived data as priors for effective kinetic parameters, and defines a scalar biofilm controllability index that interpolates between stainless-steel and liquid-infused-surface (LIS) inner liners.

The combined pipeline produces multi-objective trade-offs between illumination and available solar power, sintering energy and shell geometry, internal thermal margins, and steady-state biofilm suppression for candidate liner materials. Although the results are not intended as quantitative habitat predictions, they show that existing open planetary and microbiological datasets can be integrated into a transparent, reproducible workflow for early-stage design of lunar polar microhabitats that must coexist with, and partially control, microbial life.

1 Introduction

Future lunar surface architectures, particularly at the polar regions, must simultaneously satisfy three tightly coupled constraints: (i) power availability under highly anisotropic, seasonally modulated illumination; (ii) thermal management in the presence of steep radiative gradients and permanently shadowed regions (PSRs); and (iii) mechanical robustness and constructability using in-situ resources. Long-baseline simulations with LOLA topography have shown that only a small fraction of polar terrain achieves near-continuous or quasi-continuous illumination over a precessional cycle [1, 2], while Diviner radiometry reveals strong spatial variability in regolith thermophysical properties that governs diurnal and seasonal temperature swings [3]. These constraints are particularly acute along narrow ridges and crater rims where illumination, access to PSRs, and engineering feasibility must be traded against each other. In parallel,

the NASA Open Science Data Repository (OSDR) has emerged as an open-science model for spaceflight biology, curating space-flown and ground-based analogue datasets and demonstrating how multiomics, physiological, and imaging data from low-sample-size missions can be integrated to advance biofilm biology and enable AI- and ML-based analysis pipelines [4].

In-situ resource utilization (ISRU) has progressed from conceptual studies to experimental demonstrations of solar-driven sintering and additive manufacturing with lunar regolith simulants. Concentrated sunlight has been used to fabricate three-dimensional bricks and structural elements by directly sintering regolith layers, without imported binders [5]. Recent reviews of lunar regolith forming technologies emphasize that sintered or cast regolith could provide radiation shielding, structural shells, and thermal mass for surface infrastructure, provided that process windows and mechanical properties can be reliably controlled under lunar conditions [6]. These developments motivate microhabitat concepts in which thin, sintered shells act as structural and radiative envelopes, while internal liners and inserts host functional subsystems.

Against this background, microbial life is not an optional add-on but an intrinsic component of any long-duration human presence in space. Biofilms—surface-associated, matrix-embedded microbial communities—are now recognized as a dominant mode of microbial life on Earth and an important contributor to the early fossil record [4]. In the context of spaceflight, environmental surveys and longitudinal studies at the ISS have shown that confined habitats develop characteristic microbiomes shaped by crew, materials, and operational patterns [7, 8]. Reviews focused on biofilms in space highlight that biofilm formation can compromise hardware, alter corrosion and fouling of stainless-steel components, and modulate health risks for the crew, while also offering potential benefits for bioprocesses and life-support functions [9, 10].

Targeted spaceflight experiments have begun to probe biofilm formation under microgravity and spacecraft-relevant conditions. The Space Biofilms project and the Bacterial Adhesion and Corrosion (BAC) investigations examine how mono- and polymicrobial biofilms grow on stainless steel, respond to microgravity, and interact with candidate biocides and surface treatments, using organisms such as *Pseudomonas aeruginosa* and *Penicillium rubens* as model systems [11, 12]. These datasets provide quantitative constraints on attachment rates, extracellular polymeric substance (EPS) production, stress-response pathways, and material-specific differences in colonization that are directly relevant to engineered surfaces in space habitats.

The present work explores a deliberately constrained problem at the intersection of these domains: the design space of sintered-regolith microhabitats whose internal surfaces serve as controlled interfaces for biofilm formation and suppression. Rather than proposing a full-scale habitat architecture, we focus on a single class of geometries—spherical-cap sandwich shells—anchored to polar ridge sites with favorable illumination and regolith properties. We then couple: (i) a generative geometric model that produces a family of shells parameterized by radius, height, and thickness; (ii) a one-dimensional radial conduction model that evaluates internal climate under polar forcing; and (iii) a reduced biofilm model that maps illumination-derived UV doses and material properties into steady-state biofilm biomass and a scalar controllability index.

The goal is not to produce operational design recommendations, but to construct a computational scaffold that links open planetary and microbiological measurements to architecturally legible variables. By explicitly tracing the mapping from LOLA/Diviner/LROC products and ISS biofilm experiments into effective biofilm and thermo-physical parameters, we aim to clarify how far one can go, using existing open data and modest computational resources, in treating biofilm dynamics as a first-class design driver for lunar microhabitats, rather than as an afterthought or purely qualitative risk factor.

2 Methods

2.1 Geometry and Generative Model

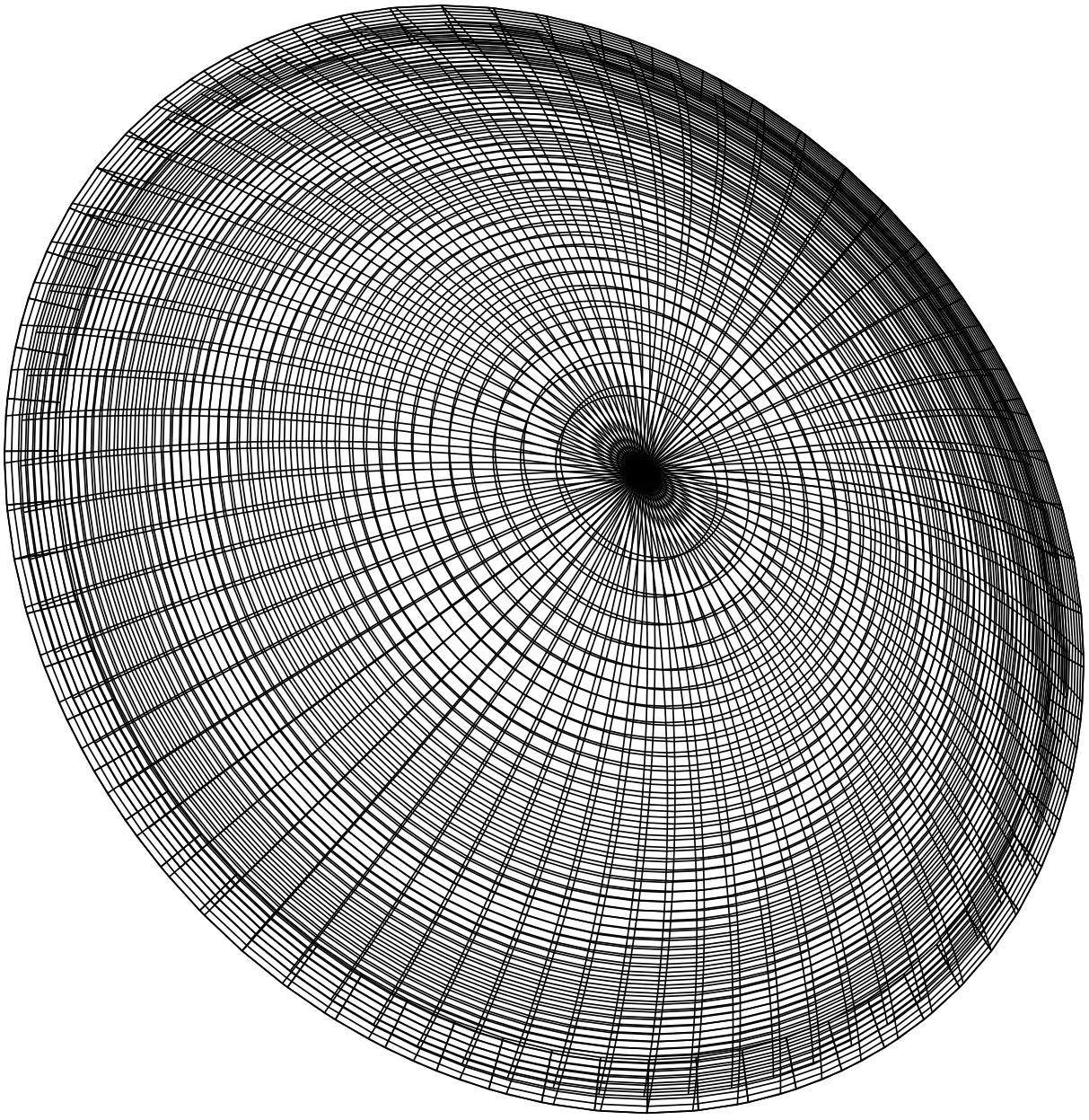


Figure 1: Lunar Shell (Code: `lunar_shell.htm`).

We represent the microhabitat as a spherical-cap sandwich shell characterized by base radius a, outer radius R, rise h, and thickness t (Figure 1). The exterior surface is a spherical cap of radius

$$S_o = \frac{R^2 + h^2}{2h}, \tag{1}$$

with center located a distance $S_o - h$ below the base plane and truncated at polar angle

$$\theta_{\max} = \arcsin \frac{a}{S_o}. \tag{2}$$

The interior surface is offset inward by t, yielding an inner sphere of radius

$$S_i = \frac{(R-t)^2 + (h-t)^2}{2(h-t)}. \tag{3}$$

This parametrization is consistent with classical spherical-cap dome and shell geometries used in structural shell theory [13, 14].

The rim is modeled as a rectangular annulus of width w in the base plane that provides a locally planar interface to the surrounding regolith and accommodates mechanical tie-downs. A fillet of width w_r blends the cap into the rim, avoiding sharp curvature discontinuities that would be structurally unfavorable and difficult to realize with sintered regolith. The generative model exposes (R, h, t, w, w_r) as primary geometric parameters and uses a dimensionless discretization (M, N) for meridians and parallels. The outer and inner surfaces are sampled as

$$\theta_m = m\frac{\theta_{\max}}{M}, \quad m = 0, \ldots, M, \tag{4}$$

$$\phi_n = n\frac{2\pi}{N}, \quad n = 0, \ldots, N-1, \tag{5}$$

with corresponding Cartesian coordinates

$$\mathbf{x}_{m,n}^{(o)} = \begin{bmatrix} S_o \sin\theta_m \cos\phi_n \\ S_o \sin\theta_m \sin\phi_n \\ S_o \cos\theta_m - (S_o - h) \end{bmatrix}, \tag{6}$$

$$\mathbf{x}_{m,n}^{(i)} = \begin{bmatrix} S_i \sin\theta_m \cos\phi_n \\ S_i \sin\theta_m \sin\phi_n \\ S_i \cos\theta_m - (S_i - h + t) \end{bmatrix}. \tag{7}$$

The discretized shell is exported as a watertight triangulated OBJ for downstream thermo-physical modeling and as a wireframe for interactive inspection. The spherical-cap abstraction deliberately ignores local stiffeners, penetrations, and secondary structures, in line with early-stage shell idealizations in the shell and dome literature [13, 14].

For thermo-physical simulations, the 3D shell is collapsed to an effective one-dimensional radial stack. The sandwich is interpreted as a sequence of concentric layers with effective radii r_i and properties $(k_i, \rho_i, c_{p,i}, \epsilon_i)$, where k_i is the thermal conductivity, ρ_i the density, $c_{p,i}$ the specific heat, and ϵ_i the emissivity. The inner radius r_0 and outer radius r_L are chosen to match the physical thickness t, and layer thicknesses Δr_i follow either a uniform or bias-refined partition, concentrating resolution where thermal gradients are largest (typically near the exterior surface). This reduction is consistent with standard treatments of heat conduction in thin spherical shells, where in-plane conduction is negligible compared to radial conduction under slowly varying boundary conditions [15, 16].

2.2 Site Screening and Horizon-Based Polar Illumination

Let $z(\mathbf{x})$ denote local DEM elevations (LOLA polar stereographic tiles). For a candidate center cell $\mathbf{x}_0$, the azimuthal horizon elevation $\eta(\psi)$ is

$$\eta(\psi) = \max_{r \in [r_{\min}, r_{\max}]} \arctan \frac{z(\mathbf{x}_0 + r\hat{\mathbf{e}}_\psi) - z(\mathbf{x}_0)}{r}, \tag{8}$$

where $\hat{\mathbf{e}}_\psi = (\cos\psi, \sin\psi)$, and $r_{\min}$, $r_{\max}$ define an inner and outer search radius (e.g. 100 m–10 km). We discretize $\psi \in [0, 2\pi)$ into N_ψ bins and r into N_r steps. For each (ψ_j, r_k) we compute the

finite-difference slope $(\Delta z)/r$ and store the maximum per azimuth, yielding

$$\eta_j = \max_k \arctan \frac{z(\mathbf{x}_0 + r_k \hat{\mathbf{e}}_{\psi_j}) - z(\mathbf{x}_0)}{r_k}, \quad j = 1, \ldots, N_\psi. \tag{9}$$

This "radial maximum-slope" horizon construction follows the horizon-based illumination methods used in polar illumination studies and toolchains such as IllumNG [1, 17–20].

2.3 Approximate polar solar track.

Near the lunar poles, the solar elevation above the local horizon is small and varies slowly over a lunation and precessional cycle. For a site at latitude ϕ and longitude λ, we approximate the solar elevation $\alpha(t)$ over a lunation as

$$\alpha(\psi) \approx |90^\circ - |\phi|| - \varepsilon \cos(\psi + \phi_s), \tag{10}$$

where ψ is the local azimuth, $\varepsilon \approx 1.54^\circ$ is an effective obliquity parameter, and ϕ_s is a phase capturing the alignment between the Sun's declination and site longitude. This reduced-form model neglects libration and higher-order orbital effects but is sufficient for ranking sites by fractional illumination f_{illum} in a manner qualitatively consistent with long-baseline simulations using full ephemerides and high-resolution DEMs [1, 2, 18].

Given $\eta(\psi)$ and $\alpha(\psi)$, the Sun is visible at azimuth ψ if

$$\alpha(\psi) > \eta(\psi) + \delta, \tag{11}$$

where δ is a small safety margin (e.g. 0.1°) that accounts for DEM uncertainty, interpolation artifacts, and the finite solar angular size [19, 20]. We define a binary visibility function

$$v(\psi) = \begin{cases} 1, & \alpha(\psi) > \eta(\psi) + \delta, \\ 0, & \text{otherwise.} \end{cases} \tag{12}$$

To approximate the fraction of a precession-modulated cycle during which the site is illuminated, we average $v(\psi)$ over azimuth and over a discrete set of seasonal phases θ_q:

$$f_{\text{illum}} = \frac{1}{QN_\psi} \sum_{q=1}^{Q} \sum_{j=1}^{N_\psi} v(\psi_j; \theta_q), \tag{13}$$

where θ_q parameterizes the slow variation in apparent solar elevation due to orbital precession over several years. In practice we evaluate $\alpha(\psi; \theta_q)$ for a small set of phases (e.g. $Q = 8$) spanning the precession cycle, mirroring the horizon-database strategies used in recent polar illumination analyses [1, 17, 18].

2.4 Implementation and computational scope.

We restrict horizon calculations to a small set of candidate pixels per region, pre-filtered by slope and local relief (e.g. pixels with $s < 10^\circ$ and local maxima within a $k \times k$ neighborhood). For each candidate we compute $\eta(\psi)$ and f_{illum} using a pure-CPU implementation that scales as $\mathcal{O}(N_\psi N_r)$ per site. For a DEM cutout with $N \sim 10^3$ and $N_\psi \sim 720$, $N_r \sim 500$, this allows tens of candidate sites to be evaluated on a standard workstation, comparable to CPU-based implementations used in recent horizon-database approaches for polar site screening [18–20].

2.5 NASA Open Datasets and Derived Products

We use three primary Lunar Reconnaissance Orbiter (LRO) products: (i) Lunar Orbiter Laser Altimeter (LOLA) gridded DEMs, (ii) Diviner Lunar Radiometer Experiment (DLRE) gridded thermophysical products, and (iii) Lunar Reconnaissance Orbiter Camera (LROC) imagery for local context. All are accessed via NASA's Planetary Data System and Open Science Data Repository.

2.5.1 Topography (LOLA).

Global and polar DEMs from LOLA Gridded Data Records (GDRs) are used to derive slopes, local relief, and horizons around candidate polar ridges. We work with polar stereographic tiles at nominal resolutions of 20–30 m/pixel, consistent with previous polar illumination studies [1, 2, 20]. For each candidate region we subset a local window (typically $\leq 25 \times 25\,\mathrm{km}$) centered on the site, optionally downsampled to $\leq 30\,\mathrm{m}$ via block averaging, and compute $\eta(\psi)$ using the radial maximum-slope construction described in §2.2. This maintains CPU and memory usage at modest levels while preserving the geometric fidelity needed for site ranking. The approach follows long-baseline polar illumination analyses that compute horizon elevations along dense azimuthal grids, but here it is restricted to a coarse grid and a reduced set of candidate ridge pixels appropriate for early-stage architectural screening [1, 2, 18, 20].

2.5.2 Thermophysics (Diviner/DLRE).

DLRE GDRs provide multi-channel solar reflectance and thermal emission products, including bolometric albedo, emissivity, and temperature statistics (mean, minimum, maximum, standard deviation) over diurnal and seasonal timescales. Our use of Diviner-derived bolometric albedo and thermal inertia as proxies for regolith thermophysical properties follows Hayne et al. [3], who derived global maps of upper-regolith thermal inertia from DLRE multi-spectral observations. These maps have been widely used to constrain 1D and 3D thermal models of near-surface temperatures on airless bodies [16, 21]. We use Diviner products to constrain effective surface parameters (α, ϵ) and to validate 1D conduction models against observed temperature ranges at the selected sites. Rather than fully co-registering DLRE rasters to LOLA at the pixel level, we sample Diviner grid cells at or around the ROI center coordinates and aggregate them into site-level thermophysical parameters. This is sufficient for parameterizing the external boundary conditions of the microhabitat shells at the level of fidelity targeted here.

2.5.3 Imagery and local context (LROC).

Narrow-angle camera (NAC) frames provide meter-scale context for assessing boulder fields, small craters, and surface texture near candidate microhabitat sites. We use NAC subframes to verify that the local slopes and relief inferred from LOLA are not dominated by artifacts or unresolved features and to assess terrain suitability for robotic emplacement and sintering (e.g. avoidance of large boulders or fractured blocks). Where available, LROC slope products and controlled mosaics are used for verification and high-resolution adjustments, following standard workflows in polar site selection studies [2, 20].

2.5.4 Local polar preprocessing pipeline.

For each candidate polar ROI we apply a small, CPU-only preprocessing pipeline:

1. subset a LOLA DEM (GeoTIFF [22, 23]) into an $N \times N$ array z_{ij} covering a small polar cutout ($< 25 \times 25\,\mathrm{km}$, optionally downsampled to $\leq 30\,\mathrm{m}$); export `.npy` and `.csv` (georeferenced index) using `polar_site_preprocess.py`;

2. compute $\eta(\psi)$ with `illumination.py` and the polar solar track to obtain f_{illum} and the binary visibility curve for each candidate point; store in `.npz`;
3. sample co-located DLRE GDR grid cells (bolometric albedo, ϵ, and representative temperature statistics) at or around the ROI center coordinates; aggregate to site-level thermophysical parameters and export JSON with metadata (source, date, native resolution).

Integrable outputs are: (i) slope rasters and $s < 10^\circ$ masks; (ii) tables of f_{illum} and horizon visibility metrics for candidate sites; and (iii) a simple ranking showing that ridges at $|\phi| \gtrsim 89^\circ$ with clean azimuthal sky sectors maintain $f_{\mathrm{illum}} \gtrsim 0.7$ over the precession-modulated seasonal phase, consistent with Mazarico et al. [1], Gläser et al. [2], Zhang et al. [18].

2.6 Radial Thermal Model and Internal Climate

We model transient heat conduction in concentric layers $i = 1, \ldots, L$ representing the outer sintered shell, bulk regolith, liner, and internal protective layers. Let $r \in [r_0, r_L]$ denote the radial coordinate across the shell thickness. The temperature $T_i(r, t)$ in layer i evolves according to

$$\frac{\partial T_i}{\partial t} = \alpha_i(T) \left(\frac{\partial^2 T_i}{\partial r^2} + \frac{1}{r} \frac{\partial T_i}{\partial r} \right), \qquad r \in [r_{i-1}, r_i], \tag{14}$$

$$k_i(T) \left. \frac{\partial T_i}{\partial r} \right|_{r=r_0} = h_{\mathrm{int}}(T_{\mathrm{int}} - T_i(r_0, t)), \tag{15}$$

$$k_i(T) \left. \frac{\partial T_i}{\partial r} \right|_{r=r_L} = q_\odot(t) - \epsilon\sigma \left(T_i(r_L, t)^4 - T_{\mathrm{bg}}^4 \right), \tag{16}$$

where $\alpha_i(T) = k_i(T)/(\rho_i c_{p,i}(T))$ is the thermal diffusivity, ρ_i the density, and $c_{p,i}(T)$ the specific heat. At internal interfaces $r = r_i$ we enforce continuity of temperature and heat flux:

$$T_i(r_i, t) = T_{i+1}(r_i, t), \tag{17}$$

$$k_i(T) \left. \frac{\partial T_i}{\partial r} \right|_{r=r_i^-} = k_{i+1}(T) \left. \frac{\partial T_{i+1}}{\partial r} \right|_{r=r_i^+}. \tag{18}$$

This is the standard 1D spherical conduction equation with piecewise-constant or weakly temperature-dependent material properties, widely used in modeling regolith and near-surface temperatures on airless bodies [16, 21].

The absorbed solar term is

$$q_\odot(t) = \alpha S_\odot v_\odot(t), \tag{19}$$

where α is the bolometric albedo, $S_\odot$ the solar constant at the Moon, and $v_\odot(t)$ a visibility factor encoding both the horizon mask $v(\psi)$ and local incidence geometry over a rotation/precession cycle. Exterior radiative exchange is modeled as a linearized term about a reference temperature T_{ref} when assembling the discrete system:

$$\epsilon\sigma(T^4 - T_{\mathrm{bg}}^4) \approx h_{\mathrm{rad}}(T - T_{\mathrm{ref}}), \qquad h_{\mathrm{rad}} = 4\epsilon\sigma T_{\mathrm{ref}}^3. \tag{20}$$

This linearization is standard in regolith thermophysical models and enables implicit time-stepping while retaining the dominant nonlinearity in the energy balance [3, 16].

We discretize r into N_r nodes and apply an implicit or Crank–Nicolson scheme in time. The resulting linear systems are tridiagonal and solved on CPU using banded solvers. Material property functions $k_i(T)$ and $c_{p,i}(T)$ are parameterized from DLRE-constrained regolith behavior [3] and laboratory measurements of regolith simulants and sintered layers, including the strong

dependence of effective thermal conductivity on porosity, contact area, and temperature [15, 24]. The outermost layer represents sintered regolith with properties adjusted by a porosity factor, while deeper layers approach bulk regolith values. This level of abstraction matches that of other 1D regolith models used in planetary thermophysical studies [16, 21].

2.7 Sintering Energy and Autonomous Construction

Let m be the processed mass of regolith. We denote by E_s the specific energy (J/kg) required for densification and sintering using microwaves or concentrated solar input. The total energy is

$$E_{\text{tot}} = E_s m. \tag{21}$$

For a spherical-cap shell with outer radius R and thickness t, embedded in a processed volume of radius R_{proc}, the processed mass is

$$m = \rho_{\text{reg}} V_{\text{proc}}, \tag{22}$$

with

$$V_{\text{proc}} \approx \pi R_{\text{proc}}^2 t_{\text{proc}}, \tag{23}$$

where t_{proc} is an effective processed depth. In the absence of detailed process models, E_s and the characteristic sintering time τ are treated as effective parameters informed by experimental demonstrations of solar and microwave regolith sintering and recent reviews of lunar regolith forming technologies for in-situ construction [5, 6, 25–28]. Reported specific energies for microwave and solar sintering of regolith simulants span $\sim 3 \times 10^3$–2×10^4 J/g, depending on process efficiency, peak temperature, and microstructure [25–27, 29].

The required peak power P_{peak} and processing time τ are constrained by the available duty cycle at the selected site, which is itself a function of f_{illum}. Assuming a fraction η of incident solar power can be converted to usable sintering power through concentrators and conversion hardware,

$$P_{\text{avail}} = \eta f_{\text{illum}} A_{\text{aperture}} S_{\odot}, \tag{24}$$

and we require $P_{\text{peak}} \leq P_{\text{avail}}$ and $E_{\text{tot}} = P_{\text{peak}} \tau$. Autonomous construction scenarios then trade off $(R, t, R_{\text{proc}}, \eta, \tau)$, constrained by the illumination-driven power envelope and by process windows derived from sintering experiments [5, 25, 26, 28]. The formulation is deliberately low-dimensional but compatible with more detailed process models that resolve temperature fields, microstructural evolution, and gravity effects on sintering dynamics [25, 26].

2.8 Biofilm Attachment and Growth under Radiative Stress

We adopt a lumped, surface-attachment model coupled to Monod kinetics with a radiation-damage term:

$$\frac{dA}{dt} = k_a \, \Phi(R_a, \theta) \, C_b - k_d A - \phi(\mathcal{R}) A, \tag{25}$$

$$\frac{dB}{dt} = \mu_{\max} \frac{A}{K_s + A} B - k_d B - \phi(\mathcal{R}) B, \tag{26}$$

where A is the attached biomass (surface density), B the biofilm biomass (volume density), C_b a bulk (planktonic) concentration, k_a an attachment rate coefficient, and k_d a detachment or death rate. The function $\Phi(R_a, \theta)$ is a dimensionless attachment factor depending on surface roughness R_a and contact angle θ, modulating the effective attachment flux. $\mu_{\max}$ is the maximum specific growth rate, K_s a half-saturation constant, and $\phi(\mathcal{R})$ an effective damage rate that depends on a radiative dose parameter $\mathcal{R}$ (e.g. UV or shortwave dose on the inner lining).

This structure follows classical biofilm models that resolve attachment/detachment and Monod-type growth in low-dimensional ODE systems [30–32], but here we collapse substrate dynamics into effective parameters and focus on the mapping from material properties and radiative forcing to steady-state biomass. We assume $\Phi(R_a, \theta)$ increases with R_a and hydrophilicity up to saturation, consistent with experimental observations of biofilm formation on rough, hydrophilic stainless-steel and polymeric surfaces [33, 34], and that $\phi(\mathcal{R})$ increases monotonically with $\mathcal{R}$, representing radiative damage to cells and extracellular polymeric substances (EPS).

The dose parameter $\mathcal{R}$ subsumes local UV intensity, exposure duration, and any attenuation by the liner material, and can be related to physical UV doses ($\mathrm{mJ\,cm^{-2}}$) via a simple scaling. Threshold-like and sigmoidal dose–response behaviors encoded in $\phi(\mathcal{R})$ are consistent with UV inactivation studies on *P. aeruginosa* and mixed-species biofilms, where biofilm-bound cells exhibit higher resistance and non-linear inactivation curves relative to planktonic cells [35–39]. In the present work, $\mathcal{R}$ is treated as a dimensionless proxy linked to the polar illumination model and internal UV delivery strategy rather than as a fully calibrated physical dose.

2.9 Biological Data

We delimit a purely theoretical biofilm response model whose parameter ranges are informed, but not fully calibrated, by existing NASA and ISS datasets. The goal is to capture relative trends across materials and gravity regimes, not to reproduce any particular experiment in detail. We therefore treat ISS-derived data as defining broad priors over attachment, growth, detachment, and damage parameters for a small set of gravity–material combinations [4, 7–12].

2.9.1 Environmental metagenomics (ISS)

Shotgun metagenomes and functional gene profiles from ISS habitable surfaces and crew microbiomes provide a first layer of constraints. Longitudinal studies of the astronaut microbiome and ISS surface communities show that spaceflight induces shifts in taxa and functional genes related to adhesion, stress response, and host interaction [7, 8]. In particular, genes involved in EPS biosynthesis, quorum sensing, oxidative-stress mitigation, and metal ion homeostasis are enriched in many ISS-associated communities relative to terrestrial baselines, consistent with a strong selection for biofilm-forming potential in confined, desiccation-prone habitats.

2.9.2 Targeted biofilm experiments

NASA's Space Biofilms and Bacterial Adhesion and Corrosion (BAC) investigations probe biofilm formation by selected organisms on spacecraft-relevant materials under microgravity and controlled environmental conditions [11, 12]. These experiments grow mono- and polymicrobial biofilms (e.g. *P. aeruginosa* and *P. rubens*) on stainless steels, liquid-infused surfaces, and related coatings, and quantify biomass, morphology, corrosion, and transcriptional responses as functions of material, flow, nutrient, and gravity conditions. Reviews on biofilms and human spaceflight synthesize these and related studies, emphasizing that microgravity tends to accelerate biofilm formation on metallic and polymeric surfaces and can enhance tolerance to disinfectants [9, 10].

2.9.3 OSD-554 (Space Biofilms; *Pseudomonas aeruginosa*)

The OSD-554 study in NASA's Open Science Data Repository provides time series of biofilm biomass, confocal imaging, and transcriptomics for *P. aeruginosa* grown on stainless-steel and LIS-like coupons under ISS conditions [11]. From these experiments one can extract, in a coarse-grained manner, effective attachment rates, maximum specific growth rates, detachment rates, and UV- or oxidant-sensitivity proxies for different materials and gravity levels. In the present

work we compress these into a small number of effective model parameters $(\mu_{\max}, k_d, \Phi_0, \beta_\phi)$ for each gravity–material combination, retaining only the relative ordering across materials and the approximate spread of the distributions.

2.9.4 Local pipeline and exports

We provide a reproducible, offline correlation between ISS-derived biofilm data and the reduced model used in §2.8:

1. Select two or more ISS-derived functional marker sets (e.g. adhesion, EPS, oxidative stress, DNA repair) and define qualitative scores for high-, medium-, and low-biofilm-forming potential on each material class, using environmental surveys and targeted experiments as guides [4, 9, 10].
2. For each gravity–material pair (microgravity vs. 1g; SS-like vs. LIS-like), map these qualitative scores to a small ensemble of effective parameter tuples $(\mu_{\max}, k_d, \Phi_0, \beta_\phi)$ consistent with the observed trends in OSD-554 and BAC datasets.
3. For each prior sample, integrate the reduced biofilm model over a wide range of illumination-derived UV dose parameters $\mathcal{R}$ (defined in §2.8), and export steady-state biomasses and derived quantities as `.csv` and `.npz` for downstream trade-off analysis.

The expected outputs are structured priors over steady-state biomass and attachment metrics for inner-liner treatments that emulate SS-like and LIS-like behavior under microgravity, and a consistent mapping from site-level illumination and geometry to biofilm-relevant parameters in the reduced model. (`osd554_priors.py`)

2.10 Trade-off Structure and Dimensionless Indices

2.10.1 Illumination, Sintering Energy, and Internal Climate

We evaluate the integrated pipeline on representative polar ridge sites selected from LOLA/-Diviner data. Sites with $s < 10^\circ$ and $f_{\text{illum}} \gtrsim 0.7$ are prioritized, following earlier work that identified quasi-continuously illuminated ridges near the lunar poles [1, 2, 18]. For each site we compute:

1. f_{illum} and visibility curves from horizons and approximate solar tracks;
2. required sintering energy E_{tot} for shells with (R, h, t) spanning a design grid;
3. internal thermal performance (minimum/maximum interior temperatures, gradients across the shell).

We map these to dimensionless parameters such as

$$\Pi_{\text{illum}} = f_{\text{illum}}, \tag{27}$$

$$\Pi_{\text{sint}} = \frac{E_{\text{tot}}}{\eta A_{\text{aperture}} S_\odot \tau}, \tag{28}$$

$$\Pi_{\text{therm}} = \frac{\Delta T_{\text{shell}}}{T_{\text{int}}}, \tag{29}$$

where ΔT_{shell} is a characteristic temperature drop across the shell. Π_{illum} captures the effective duty cycle of available solar power, Π_{sint} measures the ratio between required and available sintering energy over a processing time τ, and Π_{therm} summarizes the internal climate in terms of shell temperature gradients. Sites with Π_{illum} high, $\Pi_{\text{sint}} \lesssim 1$, and Π_{therm} small are considered favorable, consistent with the qualitative trade-offs between illumination, thermal environment, and construction feasibility identified in prior polar habitat studies [1–3, 16]. (`integration.py`)

2.10.2 Biofilm Controllability Index

From the biofilm model we define a controllability index

$$\mathcal{C}_{\text{bio}} = 1 - \frac{B_\infty}{B_{\text{ref}}}, \tag{30}$$

where B_∞ is the steady-state biofilm biomass under a given liner treatment and UV/cleaning regimen, and B_{ref} is a reference biomass (e.g. worst-case SS-like, microgravity-adapted scenario). By construction $\mathcal{C}_{\text{bio}} \in [0, 1]$, with higher values indicating easier control (lower steady-state biomass). The normalization with respect to B_{ref} makes the index directly comparable across sites and liner classes and parallels normalized performance indices used in process-systems engineering and biofilm control studies [30, 37, 40]. Using prior ensembles for $\mathcal{P}_{\text{bio}} = (\mu_{\max}, k_d, \Phi_0, \beta_\phi)$, we obtain bands for $\mathcal{C}_{\text{bio}}$ under SS-like and LIS-like treatments and examine how these interact with illumination-driven dose and thermal constraints. (`biofilm_controllability_index.py`)

2.10.3 Multi-objective Site and Design Ranking

We combine illumination, energy, thermal, and biofilm indices into a composite score

$$\mathcal{S} = w_1 \Pi_{\text{illum}} - w_2 \Pi_{\text{sint}} - w_3 \Pi_{\text{therm}} + w_4 \mathcal{C}_{\text{bio}}, \tag{31}$$

with weights w_i chosen to reflect mission priorities (e.g. minimizing sintering energy vs. maximizing biofilm controllability). This yields a ranking of site–design pairs (ridge, R, h, t), which can be visualized as Pareto fronts and heatmaps over (R, t) for fixed h/R and site. The structure is typical of multi-objective trade-space explorations in space systems and habitat design, but here explicitly incorporates a biofilm controllability dimension alongside power and thermal metrics [16, 40]. (`d_uv.py`)

2.11 Software, Reproducibility, and Data

The geometry tool emits OBJ; the interactive renderer is a single-file HTML/JS viewer for inspection. Horizon and illumination calculations are implemented in Python with NumPy/SciPy; thermal solvers use 1D banded linear algebra; biofilm response models are ODE-based. All workflows are CPU-only and are designed to run on a standard desktop or laptop. Data products (LOLA/Diviner cutouts, horizon profiles, thermal transients, biofilm parameter priors) and code (including environment specifications, exact seeds, and compiled artifacts) are deposited alongside extended references and documentation [41], enabling full reproduction of the figures and indices reported here.

2.11.1 lunar_shell.htm

```
<!DOCTYPE html><html lang=en><head><meta charset=utf-8>
<meta name=viewport content="width=device-width,initial-scale=1">
<title>Lunar Regolith Shell - Parametric Canvas Renderer</title>
<style>
:root{--bg:#fff;--fg:#000;--muted:#000;--accent:#000}
html,body{height:100%;margin:0;background:var(--bg);color:var(--fg);font-family:Arial}
.wrap{display:flex;height:100%}
.panel{width:340px;padding:16px;box-sizing:border-box;border-right:1px solid #1c2430}
.panel h1{font-size:18px;margin:0 0 12px 0;color:var(--fg)}
.panel .desc{font-size:12px;color:var(--muted);line-height:1.5;margin-bottom:12px}
.row{margin:10px 0}
.row label{display:block;font-size:12px;color:var(--muted)}
.row input[type=range]{width:100%}
.row input[type=number]{width:100%;padding:6px;background:#000;color:var(--fg);
border:1px solid #1c2430;border-radius:8px}
.btn{display:inline-block;padding:8px 10px;background:#fff;color:var(--fg);border:1px solid #1c2430;
border-radius:10px;cursor:pointer;margin-right:4px}
.canvas-wrap{flex:1;position:relative}
#cv{width:100%;height:100%;display:block}
.legend{position:absolute;left:12px;bottom:12px;font-size:12px;color:var(--muted);
border:1px #000 solid;padding:8px 10px;border-radius:10px}
</style><style media=print>.panel,.legend{display:none}</style></head>
<body><div class=wrap><div class=panel><h1>Lunar Shell (Spherical-cap Sandwich)</h1>
```

```
<div class=desc>Wireframe renderer (orthographic, Z-up). Outer/inner radii and thickness define a regolith-sintered sandwich shell with a structural rim. No aesthetic design; parameters are precise and reproducible.</div>
<div class=row><label>Radius R [m] <span id=labR></span></label>
<input id=R type=range min=0.2 max=3.0 step=0.01 value=1.00></div>
<div class=row><label>Cap height h [m] <span id=labH></span></label>
<input id=H type=range min=0.05 max=1.50 step=0.01 value=0.40></div>
<div class=row><label>Thickness t [m] <span id=labT></span></label>
<input id=T type=range min=0.005 max=0.20 step=0.001 value=0.03></div>
<div class=row><label>Segments (meridian) <span id=labM></span></label>
<input id=M type=range min=8 max=256 step=1 value=72></div>
<div class=row><label>Segments (parallel) <span id=labN></span></label>
<input id=N type=range min=4 max=256 step=1 value=48></div>
<div class=row><label>Rim width w [m] <span id=labW></span></label>
<input id=W type=range min=0.01 max=0.40 step=0.005 value=0.08></div>
<div class=row><label>Rim rise wr [m] <span id=labWR></span></label>
<input id=WR type=range min=0.00 max=0.20 step=0.005 value=0.04></div>
<div class=row><label>Rotate [rad/s] <span id=labSpin></span></label>
<input id=SP type=range min=-0.8 max=0.8 step=0.1 value=0></div>
<div class=row><button class=btn id=exportBtn>Export OBJ</button>
<button class=btn id=exportSvgBtn>Export SVG</button></div>
<div class=desc>OBJ export is analytic (no textures). Inner cap computed with radius reduced by thickness t; a rim ring closes the shell.</div>
</div>
<div class=canvas-wrap><canvas id=cv></canvas>
<div class=legend>Keys: drag=rotate, scroll=zoom, dblclick=reset.</div></div>
</div>
<script>
const st={R:1.0,H:0.4,T:0.03,M:72,N:48,W:0.08,WR:0.04,spin:0.12,rotX:-0.35,rotY:0.55,zoom:1.8};
const cv=document.getElementById('cv');const ctx=cv.getContext('2d');
function resize(){const dpr=Math.max(1,window.devicePixelRatio||1);
cv.width=cv.clientWidth*dpr;cv.height=cv.clientHeight*dpr;ctx.setTransform(dpr,0,0,dpr,0,0);}
window.addEventListener('resize',resize);resize();
function uiBind(id,lab,key,fmt=(v)=>v){
  const e=document.getElementById(id),l=document.getElementById(lab);
  function upd(){st[key]=parseFloat(e.value);l.textContent='('+fmt(st[key])+')';}
  e.addEventListener('input',()=>{upd();rebuild();});
  upd();
}
uiBind('R','labR','R',v=>v.toFixed(2));uiBind('H','labH','H',v=>v.toFixed(2));
uiBind('T','labT','T',v=>v.toFixed(3));uiBind('M','labM','M',v=>v.toFixed(0));
uiBind('N','labN','N',v=>v.toFixed(0));uiBind('W','labW','W',v=>v.toFixed(3));
uiBind('WR','labWR','WR',v=>v.toFixed(3));uiBind('SP','labSpin','spin',v=>v.toFixed(2));
let dragging=false,lx=0,ly=0;
cv.addEventListener('mousedown',e=>{dragging=true;lx=e.clientX;ly=e.clientY;});
window.addEventListener('mouseup',()=> dragging=false);
window.addEventListener('mousemove',e=>{
  if(!dragging) return;
  const dx=(e.clientX-lx)/cv.clientWidth,dy=(e.clientY-ly)/cv.clientHeight;
  st.rotY+=dx*3.0;st.rotX+=dy*3.0;
  lx=e.clientX;ly=e.clientY;
});
cv.addEventListener('wheel',e=>{st.zoom *=Math.exp(-e.deltaY*0.001);});
cv.addEventListener('dblclick',()=>{st.rotX=-0.35;st.rotY=0.55;st.zoom=1.8;});
function matRY(a){const c=Math.cos(a),s=Math.sin(a);return [c,0,s,0,1,0,-s,0,c];}
function matRX(a){const c=Math.cos(a),s=Math.sin(a);return [1,0,0,0,c,-s,0,s,c];}
function mulM(a,b){
  const r=new Array(9);
  for(let i=0;i<3;i++)for(let j=0;j<3;j++){
    r[i*3+j]=0;
    for(let k=0;k<3;k++) r[i*3+j]+=a[i*3+k]*b[k*3+j];
  }
  return r;
}
function mulMV(m,v){
  return [
    m[0]*v[0]+m[1]*v[1]+m[2]*v[2],
    m[3]*v[0]+m[4]*v[1]+m[5]*v[2],
    m[6]*v[0]+m[7]*v[1]+m[8]*v[2]
  ];
}
function buildShell(R,H,T,M,N,W,WR){
  const S=(R*R+H*H)/(2*H);const z0=-(S-H);
  const vertsOuter=[],lines=[];
  for(let i=0;i<=M;i++){
    const phi=2*Math.PI*i/M;
    for(let j=0;j<=N;j++){
      const th=Math.acos(1-(H/S)*(j/N));
      const x=S*Math.sin(th)*Math.cos(phi);
      const y=S*Math.sin(th)*Math.sin(phi);
      const z=z0+S*Math.cos(th);
      if(z<-1e-6) continue;
      vertsOuter.push([x,y,z]);
    }
  }
  const Rin=Math.max(1e-4,R-T);
  const Sin=(Rin*Rin+(H-T)*(H-T))/(2*(H-T));
  const z0i=-(Sin-(H-T));
  const vertsInner=[];
  for(let i=0;i<=M;i++){
    const phi=2*Math.PI*i/M;
    for(let j=0;j<=N;j++){
      const th=Math.acos(1-((H-T)/Sin)*(j/N));
      const x=Sin*Math.sin(th)*Math.cos(phi);
      const y=Sin*Math.sin(th)*Math.sin(phi);
      const z=z0i+Sin*Math.cos(th);
      if(z<-1e-6) continue;
      vertsInner.push([x,y,z]);
    }
```

```
  }
  function pushGridLines(verts){
    const span=(N+1);
    for(let i=0;i<=M;i++){
      for(let j=0;j<N;j++){
        const a=i*span+j,b=i*span+j+1;
        if(verts[a]&&verts[b]) lines.push([verts[a],verts[b]]);
      }
    }
    for(let j=0;j<=N;j++){
      for(let i=0;i<M;i++){
        const a=i*span+j,b=(i+1)*span+j;
        if(verts[a]&&verts[b]) lines.push([verts[a],verts[b]]);
      }
    }
  }
  pushGridLines(vertsOuter);
  pushGridLines(vertsInner);
  const rings=64;
  for(let i=0;i<rings;i++){
    const a=2*Math.PI*i/rings,b=2*Math.PI*(i+1)/rings;
    const ro=R,ri=Math.max(1e-4,R-W);
    const p1=[ro*Math.cos(a),ro*Math.sin(a),0],p2=[ro*Math.cos(b),ro*Math.sin(b),0];
    const q1=[ri*Math.cos(a),ri*Math.sin(a),WR],q2=[ri*Math.cos(b),ri*Math.sin(b),WR];
    lines.push([p1,p2]);
    lines.push([q1,q2]);
    lines.push([p1,q1]);
    lines.push([p2,q2]);
  }
  return lines;
}
function project(v,s){
  const ww=cv.clientWidth,hh=cv.clientHeight;
  return [ ww*0.5+s*v[0],hh*0.5 - s*v[1] ];
}
let lines=[];
function rebuild(){lines=buildShell(st.R,st.H,st.T,st.M|0,st.N|0,st.W,st.WR);}
rebuild();
let last=performance.now();
function frame(t){
  const dt=(t-last)/1000;last=t;
  const rot=mulM(matRY(st.rotY),matRX(st.rotX));
  if(Math.abs(st.spin)>1e-4){st.rotY+=st.spin*dt;}
  ctx.clearRect(0,0,cv.clientWidth,cv.clientHeight);
  ctx.lineWidth=1.0;
  ctx.strokeStyle='rgba(0,0,0,1)';
  ctx.beginPath();
  for(const seg of lines){
    const a=mulMV(rot,seg[0]);const b=mulMV(rot,seg[1]);
    const A=project(a,60*st.zoom);const B=project(b,60*st.zoom);
    ctx.moveTo(A[0],A[1]);ctx.lineTo(B[0],B[1]);
  }
  ctx.stroke();
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
// Export OBJ (as you already had it)
document.getElementById('exportBtn').addEventListener('click',()=>{
  const L=buildShell(st.R,st.H,st.T,st.M|0,st.N|0,st.W,st.WR);
  let obj='# Lunar shell wireframe OBJ (edges as segments)\n';const verts=[];
  for(const seg of L){verts.push(seg[0],seg[1]);}
  for(const v of verts){obj+=`v ${v[0].toFixed(6)} ${v[1].toFixed(6)} ${v[2].toFixed(6)}\n`;}
  for(let i=0;i<verts.length;i+=2){obj+=`l ${i+1} ${i+2}\n`;}
  const blob=new Blob([obj],{type:'text/plain'});const a=document.createElement('a');
  a.href=URL.createObjectURL(blob);a.download='lunar_shell_wire.obj';a.click();
});
// Export SVG of the current view
document.getElementById('exportSvgBtn').addEventListener('click',()=>{
  const ww=cv.clientWidth,hh=cv.clientHeight;
  const rot=mulM(matRY(st.rotY),matRX(st.rotX));
  const scale=60*st.zoom;
  let svgLines='';
  for(const seg of lines){
    const a=mulMV(rot,seg[0]);
    const b=mulMV(rot,seg[1]);
    const A=project(a,scale);
    const B=project(b,scale);
    svgLines+=`<line x1="${A[0].toFixed(2)}" y1="${A[1].toFixed(2)}" x2="${B[0].toFixed(2)}" `+
    `y2="${B[1].toFixed(2)}" stroke="black" stroke-width="0.8"/>`;
  }
  const svg=`<?xml version="1.0" encoding="UTF-8"?>\n`+
            `<svg xmlns="http://www.w3.org/2000/svg" width="${ww}" height="${hh}" `+
            `viewBox="0 0 ${ww} ${hh}">\n`+
            `<g>${svgLines}</g>\n</svg>\n`;
  const blob=new Blob([svg],{type:'image/svg+xml'});
  const a=document.createElement('a');
  a.href=URL.createObjectURL(blob);
  a.download='lunar_shell_wire.svg';
  a.click();
  setTimeout(()=>URL.revokeObjectURL(a.href),1000);
});
</script></body></html>
```

2.11.2 polar_site_preprocess.py

```
#!/usr/bin/env python3
# Local polar preprocessing:
# - Load a cutout of LDEM_80S_20MPP_ADJ.TIF in https://pgda.gsfc.nasa.gov/products/90
# - Compute slopes and select candidate pixels (s<10deg)
# - Compute azimuthal horizon and illumination fraction f_illum for a few candidates
import numpy as np
import rasterio
from rasterio.windows import Window
from typing import Tuple
from illumination import radial_horizon, polar_solar_track, illumination_fraction
# Input parameters (adjust to your environment)
# Local base path where input/output files are located
BASE_PATH="/home/a/Downloads/architecture/"
# Local path to the DEM you downloaded
LDEM_PATH=BASE_PATH+"LDEM_80S_20MPP_ADJ.TIF"
# Approximate central latitude of the site (deg, negative in southern hemisphere)
SITE_LAT_DEG=-89.0   # generic example; adjust according to ROI
# Cutout size (in pixels)
CUTOUT_SIZE=1024  # 1024x1024 pixels ~ 20 km x 20 km at 20 m/px
# Pixel coordinates of the cutout center
# You obtain these by inspecting the DEM in QGIS or similar
# (row,column) in the index system of the original raster
CENTER_ROW=20000    # EXAMPLE: ADJUST
CENTER_COL=20000    # EXAMPLE: ADJUST
# Radial search range for the horizon (in pixels)
MAX_RANGE_PIX=512  # ~ 10 km for dx=20 m
# Azimuthal discretization
N_AZ=720
# Utilities
def compute_slope(dem:np.ndarray,dx:float,dy:float)->np.ndarray:
    # Slope map (in radians) computed from the DEM.
    # Gradient in meters per meter
    dz_dy,dz_dx=np.gradient(dem,dy,dx)  # dy:axis 0,dx:axis 1
    slope_rad=np.arctan(np.hypot(dz_dx,dz_dy))
    return slope_rad
def extract_cutout(path:str,center_row:int,center_col:int,
                   size:int)->Tuple[np.ndarray,rasterio.Affine]:
    # Extract a square cutout size x size centered at (row,col).
    half=size // 2
    row_off=center_row-half
    col_off=center_col-half
    with rasterio.open(path) as src:
        window=Window(col_off=col_off,row_off=row_off,width=size,height=size)
        dem=src.read(1,window=window)
        transform=src.window_transform(window)
    return dem,transform
def save_dem_cutout(dem:np.ndarray,transform:rasterio.Affine,out_prefix:str="polar_cutout"):
    """Save DEM cutout as .npy + .csv (with x,y,z)."""
    np.save(f"{BASE_PATH}{out_prefix}.npy",dem)
    # Export CSV with X,Y,Z coordinates (in meters, polar stereographic projection)
    H,W=dem.shape
    xs=np.arange(W)
    ys=np.arange(H)
    xv,yv=np.meshgrid(xs,ys)
    # Convert pixel indices to spatial coordinates
    # x=col * a + x0,y=row * e + y0 (a,e,y0,x0 from transform)
    x_coords,y_coords=rasterio.transform.xy(transform,yv,xv)
    x_coords=np.array(x_coords)
    y_coords=np.array(y_coords)
    # Save to CSV (row,col,x,y,z)
    import csv
    H,W=dem.shape
    with open(f"{BASE_PATH}{out_prefix}.csv","w",newline="") as f:
        writer=csv.writer(f)
        writer.writerow(["row","col","x_m","y_m","z_m"])
        for r in range(H):
            for c in range(W):
                writer.writerow([r,c,x_coords[r,c],y_coords[r,c],dem[r,c]])
def main():
    # Load cutout
    dem,transform=extract_cutout(LDEM_PATH,center_row=CENTER_ROW,center_col=CENTER_COL,size=CUTOUT_SIZE)
    # Spatial resolution (m/pixel). We assume square pixels.
    dx=transform.a        # pixel size in X (m)
    dy=-transform.e       # transform.e is usually negative
    print(f"DEM cutout shape:{dem.shape},dx={dx:.2f} m,dy={dy:.2f} m")
    # Compute slope map
    slope_rad=compute_slope(dem,dx=dx,dy=dy)
    slope_deg=np.degrees(slope_rad)
    # Mask of smooth pixels (e.g. s<10deg)
    smooth_mask=slope_deg<10.0
    # Select a small set of candidates
    # Example: sample 5x5 points uniformly where the mask is True
    candidate_indices=np.argwhere(smooth_mask)
    if candidate_indices.size==0:
        raise RuntimeError("There are no pixels with slope<10deg in the cutout.")
    # Select N candidates at random (for example 20)
    N_CAND=min(20,candidate_indices.shape[0])
    rng=np.random.default_rng(42)
    sel=rng.choice(candidate_indices.shape[0],size=N_CAND,replace=False)
    candidates=candidate_indices[sel]
    # Precompute approximate polar solar track
    solar_alpha=polar_solar_track(lat_deg=SITE_LAT_DEG,n_az=N_AZ)
    # For each candidate, compute horizon and illumination fraction
    results=[]
    for idx,(r0,c0) in enumerate(candidates):
        az,hor=radial_horizon(dem,dx=dx,center=(int(r0),int(c0)),n_az=N_AZ,max_range=MAX_RANGE_PIX)
```

```
        # Ensure size consistency
        if hor.shape[0] != solar_alpha.shape[0]:
            raise ValueError("Shape mismatch between horizon and solar track.")
        f_illum=illumination_fraction(horizon_elev=hor,solar_alpha=solar_alpha)
        results.append((int(r0),int(c0),float(f_illum)))
        print(f"[{idx+1:02d}/{N_CAND}] pixel=({r0},{c0}) f_illum={f_illum:.3f}")
    # Save results to CSV for later analysis
    import csv
    with open(f"{BASE_PATH}candidate_illumination.csv","w",newline="") as f:
        writer=csv.writer(f)
        writer.writerow(["row","col","f_illum"])
        writer.writerows(results)
    # Also save slope as .npy
    np.save(f"{BASE_PATH}polar_cutout_slope_deg.npy",slope_deg)
if __name__=="__main__":
    main()
```

2.11.3 illumination.py

```
#!/usr/bin/env python3
# Polar illumination estimator (horizon-based).
# Computes azimuthal horizon elevation from a DEM and estimates the fraction of
# time the Sun clears the horizon, given an approximate polar solar elevation track. NumPy-only.
# Default demo parameters are chosen to be consistent with a LOLA-like polar cutout
# (dx ~ 30 m, window ~ 15 km) and a high-latitude ridge site (|psi|>=89deg) as in the manuscript.
import numpy as np
from typing import Tuple
def radial_horizon(
    dem:np.ndarray,dx:float,center:Tuple[int,int],n_az:int=720,max_range:int|None=None,
)->Tuple[np.ndarray,np.ndarray]:
    # Return (azimuths_rad, horizon_elev_rad) of size n_az.
    # dem: 2D array of elevations [m].
    # dx: pixel spacing [m].
    # center: (row, col) index of candidate site.
    # n_az: number of azimuth samples over [0, 2pi).
    # max_range: maximum number of radial steps; if None, defaults to
    # min(10 km/dx, 0.5*min(H,W)) in pixels.
    H,W=dem.shape
    r0,c0=center
    if max_range is None:
        # cap search radius at ~10 km and at half the window size
        max_px_10km=int(10_000.0/dx)
        max_range=int(min(max_px_10km,0.5*min(H,W)))
    az=np.linspace(0.0,2*np.pi,n_az,endpoint=False)
    hor=np.full(n_az,-np.inf,dtype=float)
    for k,a in enumerate(az):
        s=np.sin(a)
        c=np.cos(a)
        r,cc=float(r0),float(c0)
        z0=float(dem[r0,c0])
        max_slope=-1e9
        for step in range(1,max_range):
            # image row axis downward
            r+=-s
            cc+=c
            ri,ci=int(round(r)),int(round(cc))
            if ri<0 or ri>=H or ci<0 or ci>=W:
                break
            dz=float(dem[ri,ci])-z0
            dist=step*dx
            slope=dz/max(dist,1e-6)
            if slope>max_slope:
                max_slope=slope
        hor[k]=np.arctan(max_slope) if np.isfinite(max_slope) else -np.pi/2
    return az,hor
def illumination_fraction(
    horizon_elev:np.ndarray,solar_alpha:np.ndarray,safety_margin_deg:float=0.1,
)->float:
    # Fraction of azimuthal cycle where solar apparent elevation alpha exceeds the horizon plus
    # a small safety margin (delta).
    # horizon_elev: horizon elevation eta(psi) [rad].
    # solar_alpha: apparent solar elevation alpha(psi) [rad].
    # safety_margin_deg: delta in degrees, accounts for DEM uncertainty and finite solar angular size.
    if horizon_elev.shape!=solar_alpha.shape:
        raise ValueError("Shape mismatch between horizon and solar arrays")
    delta=np.radians(safety_margin_deg)
    visible=(solar_alpha>(horizon_elev+delta)).astype(float)
    return float(visible.mean())
def polar_solar_track(
    lat_deg:float,n_az:int=720,axial_tilt_deg:float=1.54,seasonal_offset_deg:float=0.0,
)->np.ndarray:
    # Approximate apparent solar elevation alpha(psi) for lunar polar latitudes.
    # alpha(psi) approx |90deg-|lat|| - epsilon cos(psi+phi_s)
    # lat_deg: latitude in degrees (use |lat|>=89 for polar ridges).
    # n_az: number of azimuth samples over [0, 2pi).
    # axial_tilt_deg: effective obliquity epsilon [deg], default 1.54deg.
    # seasonal_offset_deg: phase psi_s [deg], representing precessional phase.
    # Returns alpha(psi) in radians, sampled over [0, 2pi).
    lat=np.radians(lat_deg)
    eps=np.radians(axial_tilt_deg)
    psi=np.linspace(0.0,2*np.pi,n_az,endpoint=False)
    base=abs(0.5*np.pi-abs(lat))
```

```
    alpha=base-eps*np.cos(psi+np.radians(seasonal_offset_deg))
    return alpha
def demo_fraction_from_dem(
    dem:np.ndarray,dx:float,center:Tuple[int,int],lat_deg:float,n_az:int=720,
    safety_margin_deg:float=0.1,
)->float:
    # Convenience wrapper:horizon -> polar solar track -> illumination fraction.
    _,hor=radial_horizon(dem,dx,center,n_az=n_az)
    alpha=polar_solar_track(lat_deg,n_az=n_az)
    return illumination_fraction(hor,alpha,safety_margin_deg=safety_margin_deg)
if __name__=="__main__":
    # Synthetic polar ridge demo consistent with the manuscript:
    #-Window ~ 15.36 km x 15.36 km at 30 m/pixel (H=W=512).
    #-Simple Gaussian ridge centered in x,uniform in y.
    #-High-latitude site at |psi| approx 89.7deg (quasi-continuous illumination regime).
    #-n_az=720 azimuth samples (0.5deg resolution).
    H,W=512,512
    dx=30.0  # [m] LOLA-like polar DEM resolution
    yy,xx=np.mgrid[0:H,0:W]
    # Gaussian ridge in x-direction,amplitude ~ 100 m,width ~ 1 km
    ridge_height=100.0  # [m]
    ridge_sigma_px=1_000.0/dx  # 1 km/30 m approx 33 px
    ridge=ridge_height*np.exp(-((xx-W*0.5)**2)/(2.0*ridge_sigma_px**2))
    dem=ridge
    center=(H//2,W//2)
    # Representative high-latitude ridge:|psi| approx 89.7deg
    lat_deg=89.7
    f=demo_fraction_from_dem(dem,dx,center,lat_deg=lat_deg,n_az=720,safety_margin_deg=0.1,)
    print(f"Latitude {lat_deg:.1f}deg: estimated illumination fraction approx {f:.3f}")
```

2.11.4 osd554_priors.py

```
#!/usr/bin/env python3
# https://osdr.nasa.gov/bio/repo/data/studies/OSD-554
# Biological priors derived from GLDS-554 Normalized_counts (*.xlsx).
# Uses:
# - Normalized_counts/SS_day3_bygravity.xlsx
# - Normalized_counts/LIS_day3_bygravity.xlsx
# Each file contains:
# - first column:gene_id (e.g. PA14_00010)
# - columns F...:flight/micro replicates
# - columns G...:ground/1g replicates
# Output:
# - P_bio_OSD554_priors_from_normalized.csv
# with one row per condition (micro_SS,ground_SS,micro_LIS,ground_LIS)
# and columns: condition_id,gravity,material,tolerance_score,mu_max,k_d,Phi_0,beta_phi,
# top_genes (semicolon-separated list)
import numpy as np
import pandas as pd
from pathlib import Path
from typing import List,Dict,Tuple,Optional
# GLOBAL PARAMETERS
# Directory where GLDS-554_rna-seq_Normalized_counts.zip was extracted
NORM_DIR=Path("/home/a/Downloads/architecture/")
SS_FILE=NORM_DIR/"SS_day3_bygravity.xlsx"
LIS_FILE=NORM_DIR/"LIS_day3_bygravity.xlsx"
# IMPORTANT: Number of top-|log2FC| genes per comparison if no manual list is provided
TOP_N=10
# IMPORTANT: Optional specific gene list; if empty, automatic TOP_N selection is used
CUSTOM_GENES:List[str]=[]  # e.g. ["PA14_00010","PA14_00020"]
# UTILITIES
def load_bygravity_table(path:Path)->pd.DataFrame:
    # Loads a *_day3_bygravity.xlsx file.
    # Assumes:
    # - first column = gene_id,
    # - remaining columns = samples (e.g. F16.1,G16.1,...)
    # Returns a DataFrame indexed by gene_id with expression columns.
    if not path.exists():
        raise FileNotFoundError(f"File not found:{path}")
    df=pd.read_excel(path)
    # First column = gene ID
    gene_col=df.columns[0]
    df=df.set_index(gene_col)
    # Ensure numeric type in expression columns
    df=df.apply(pd.to_numeric,errors="coerce")
    return df
def split_micro_ground(df:pd.DataFrame)->Tuple[List[str],List[str]]:
    # Separates flight (micro) and ground columns by prefix.
    # Heuristic:
    # - columns starting with 'F' -> micro
    # - columns starting with 'G' -> ground
    # Returns (micro_cols,ground_cols).
    micro_cols=[c for c in df.columns if isinstance(c,str) and c.startswith("F")]
    ground_cols=[c for c in df.columns if isinstance(c,str) and c.startswith("G")]
    if not micro_cols or not ground_cols:
        raise ValueError(
            f"Could not identify micro/ground columns; "
            f"micro={micro_cols},ground={ground_cols}"
        )
    return micro_cols,ground_cols
def compute_log2fc(df:pd.DataFrame,micro_cols:List[str],
                   ground_cols:List[str],eps:float=1e-6)->pd.Series:
```

```
    # Mean log2 fold-change (micro vs ground) per gene.
    # log2FC=log2(mean(expr_micro)+eps)-log2(mean(expr_ground)+eps).
    micro_mean=df[micro_cols].mean(axis=1)
    ground_mean=df[ground_cols].mean(axis=1)
    log2fc=np.log2(micro_mean+eps)-np.log2(ground_mean+eps)
    log2fc.name="log2FC_micro_vs_ground"
    return log2fc
def select_genes(df:pd.DataFrame,log2fc:pd.Series,
                 top_n:int,custom_genes:Optional[List[str]]=None)->List[str]:
    # Selects genes for calculating the 'tolerance_score'.
    # -If custom_genes is not None and not empty:
    #    use the intersection between custom_genes and df.index.
    # -Otherwise:
    #    sort genes by |log2FC| descending and take top_n.
    if custom_genes:
        present=[g for g in custom_genes if g in df.index]
        if not present:
            raise ValueError("None of the CUSTOM_GENES are present in the table.")
        return present
    # Automatic selection by |log2FC|
    selected=log2fc.abs().sort_values(ascending=False).head(top_n).index.tolist()
    return selected
def tolerance_score(df:pd.DataFrame,genes:List[str],cols:List[str])->float:
    # Simple 'tolerance score':
    # average normalized expression across selected genes and given columns.
    # score=mean_{g in genes,c in cols}( expr[g,c] )
    sub=df.loc[genes,cols]
    return float(sub.mean().mean())
# MAPPING SCORE->EFFECTIVE PARAMETERS
def map_scores_to_params(score_dict:Dict[str,float])->pd.DataFrame:
    # Receives a dict: condition_id->tolerance_score
    # Returns DataFrame with columns:
    # condition_id,tolerance_score,mu_max,k_d,Phi_0,beta_phi
    # using a simple monotonic mapping on the z-score of tolerance_score.
    rows=[]
    cond_ids=list(score_dict.keys())
    scores=np.array([score_dict[cid] for cid in cond_ids],dtype=float)
    # z-score normalization
    mean_s=scores.mean()
    std_s=scores.std() if scores.std()>0 else 1.0
    z=(scores-mean_s)/std_s
    # Base parameters (adjustable)
    base_mu=0.5      # h^-1
    base_kd=0.05     # h^-1
    base_Phi0=1.0
    base_beta=1.0
    alpha_mu=0.15    # sensitivity scale for mu_max
    alpha_kd=0.5     # exponential scale for k_d
    alpha_Phi=0.5    # linear scale for Phi_0
    alpha_beta=0.3   # exponential scale for beta_phi
    for cid,s,zc in zip(cond_ids,scores,z):
        # Example of mapping:
        mu_max=base_mu * (1.0+alpha_mu * zc)
        k_d=base_kd * np.exp(-alpha_kd * zc)
        Phi_0=base_Phi0 * (1.0+alpha_Phi * zc)
        beta_phi=base_beta * np.exp(-alpha_beta * zc)
        rows.append({
            "condition_id":cid,"tolerance_score":s,
            "mu_max":mu_max,"k_d":k_d,
            "Phi_0":Phi_0,"beta_phi":beta_phi
        })
    df=pd.DataFrame(rows)
    return df
# MAIN PIPELINE
def build_priors_from_normalized(top_n:int=TOP_N,
                                 custom_genes:Optional[List[str]]=None)->pd.DataFrame:
    # Builds priors from SS_day3_bygravity.xlsx and LIS_day3_bygravity.xlsx.
    # Returns DataFrame with rows: micro_SS,ground_SS,micro_LIS,ground_LIS
    # plus effective parameter columns.
    # Dictionary:condition_id->(gravity,material)
    meta_cond={
        "micro_SS":("microgravity","SS"),"ground_SS":("ground","SS"),
        "micro_LIS":("microgravity","LIS"),"ground_LIS":("ground","LIS")
    }
    # SS
    ss_df=load_bygravity_table(SS_FILE)
    ss_micro_cols,ss_ground_cols=split_micro_ground(ss_df)
    ss_log2fc=compute_log2fc(ss_df,ss_micro_cols,ss_ground_cols)
    ss_genes=select_genes(ss_df,ss_log2fc,top_n=top_n,custom_genes=custom_genes)
    # scores for SS
    score_micro_SS=tolerance_score(ss_df,ss_genes,ss_micro_cols)
    score_ground_SS=tolerance_score(ss_df,ss_genes,ss_ground_cols)
    # LIS
    lis_df=load_bygravity_table(LIS_FILE)
    lis_micro_cols,lis_ground_cols=split_micro_ground(lis_df)
    lis_log2fc=compute_log2fc(lis_df,lis_micro_cols,lis_ground_cols)
    lis_genes=select_genes(lis_df,lis_log2fc,top_n=top_n,custom_genes=custom_genes)
    score_micro_LIS=tolerance_score(lis_df,lis_genes,lis_micro_cols)
    score_ground_LIS=tolerance_score(lis_df,lis_genes,lis_ground_cols)
    # Collect scores and map to parameters
    score_dict={
        "micro_SS":score_micro_SS,"ground_SS":score_ground_SS,
        "micro_LIS":score_micro_LIS,"ground_LIS":score_ground_LIS
    }
    params_df=map_scores_to_params(score_dict)
    # Add gravity,material columns
    gravity_list=[]
    material_list=[]
    for cid in params_df["condition_id"]:
```

```
        g,m=meta_cond[cid]
        gravity_list.append(g)
        material_list.append(m)
    params_df["gravity"]=gravity_list
    params_df["material"]=material_list
    # String with genes used per condition (SS/LIS may have different sets)
    # For traceability keep them separated:
    top_genes_map={"SS":";".join(ss_genes),"LIS":";".join(lis_genes)}
    cond_genes=[]
    for cid in params_df["condition_id"]:
        mat=meta_cond[cid][1]
        cond_genes.append(top_genes_map.get(mat,""))
    params_df["top_genes"]=cond_genes
    # Column order
    cols=["condition_id","gravity","material","tolerance_score","mu_max","k_d","Phi_0",
          "beta_phi","top_genes"]
    params_df=params_df[cols]
    return params_df
def main():
    priors_df=build_priors_from_normalized(top_n=TOP_N,custom_genes=CUSTOM_GENES)
    priors_df.to_csv(NORM_DIR/"P_bio_OSD554_priors_from_normalized.csv",index=False)
    print(priors_df)
if __name__=="__main__":
    main()
```

2.11.5 integration.py

```
#!/usr/bin/env python3
# Biofilm integration for OSD-554 conditions + polar site chosen by f_illum.
# Uses:
# - candidate_illumination.csv (row,col,f_illum)
# - P_bio_OSD554_priors_from_normalized.csv (condition_id,gravity,material,
#     tolerance_score,mu_max,k_d,Phi_0,beta_phi,top_genes)
# Produces:
# biofilm_timeseries.csv with A(t), B(t) for each selected condition.
import numpy as np
import pandas as pd
from pathlib import Path
from typing import Dict,Tuple,List
# GENERAL CONFIGURATION
# Project base location (ADJUST)
BASE_DIR=Path("/home/a/Downloads/architecture/").resolve()
# Paths relative to BASE_DIR
CANDIDATE_PATH=BASE_DIR/"candidate_illumination.csv"
PRIORS_PATH=BASE_DIR/"P_bio_OSD554_priors_from_normalized.csv"
# Conditions we want to compare (must exist in the column condition_id)
# Natural example with the priors we defined:micro_SS vs micro_LIS
COND_IDS:List[str]=["micro_SS","micro_LIS"]
# Time integrator parameters (in hours)
T_END_H=240.0   # total simulation duration (e.g. 10 days)
DT_H=0.1      # time step in hours
# Base parameters of the biofilm model
C_B=1.0         # planktonic cell concentration (arbitrary)
K_S=0.5         # half-saturation constant in units of A
K_A=0.05        # base adhesion coefficient
# Reference UV dose scale (dimensionless); D_uv=D_REF*f_illum
D_REF=1.0
# UTILITIES
def load_best_candidate(path:Path)->Tuple[int,int,float]:
    # Reads candidate_illumination.csv and returns the pixel with maximum f_illum.
    # Expected CSV: row,col,f_illum
    df=pd.read_csv(path)
    if "f_illum" not in df.columns:
        raise ValueError("candidate_illumination.csv must contain column 'f_illum'.")
    idx=df["f_illum"].idxmax()
    row=int(df.loc[idx,"row"])
    col=int(df.loc[idx,"col"])
    f_illum=float(df.loc[idx,"f_illum"])
    print(f"Best candidate:row={row},col={col},f_illum={f_illum:.3f}")
    return row,col,f_illum
def load_priors(path:Path,cond_ids:List[str])->Dict[str,Dict[str,float]]:
    # Loads priors from P_bio_OSD554_priors_from_normalized.csv and
    # returns a dict: cond_id->dict(mu_max,k_d,Phi_0,beta_phi)
    df=pd.read_csv(path)
    required_cols={"condition_id","mu_max","k_d","Phi_0","beta_phi"}
    if not required_cols.issubset(df.columns):
        raise ValueError(f"The priors file must contain columns {required_cols}.")
    priors_dict:Dict[str,Dict[str,float]]={}
    for cid in cond_ids:
        row=df[df["condition_id"]==cid]
        if row.empty:
            raise ValueError(f"condition_id='{cid}' not found in priors.")
        r=row.iloc[0]
        priors_dict[cid]={
            "mu_max":float(r["mu_max"]),"k_d":float(r["k_d"]),
            "Phi_0":float(r["Phi_0"]),"beta_phi":float(r["beta_phi"])
        }
    return priors_dict
def uv_dose_from_fillum(f_illum:float,
                        d_ref:float=D_REF)->float:
    # Effective UV dose (dimensionless) from f_illum. D_uv=d_ref*f_illum
    return d_ref*f_illum
```

```
def biofilm_rhs(t:float,y:np.ndarray,params:Dict[str,float],d_uv:float)->np.ndarray:
    # Derivative field for the system:
    #     dA/dt=k_a*Phi*C_b-(k_d+phi)*A
    #     dB/dt=mu_max*A/(K_s+A)*B-(k_d+phi)*B
    # where:
    #  Phi   ~ Phi_0 (effective adhesion factor),
    #  phi  =beta_phi*D_uv.
    # t:time (h) [not used explicitly in this autonomous model]
    # y:[A,B]
    # params:{mu_max,k_d,Phi_0,beta_phi}
    # d_uv:effective UV dose (constant here).
    A,B=y
    mu_max =params["mu_max"]
    k_d=params["k_d"]
    Phi_0=params["Phi_0"]
    beta_phi=params["beta_phi"]
    # Adhesion factor; here taken constant=Phi_0
    Phi=Phi_0
    # Effective radiative damage rate
    phi_rate=beta_phi*d_uv
    # Prevention of problematic divisions
    denom=K_S+max(A,0.0)
    dA_dt=K_A*Phi*C_B-(k_d+phi_rate)*A
    dB_dt=mu_max*(A/denom)*B-(k_d+phi_rate)*B
    return np.array([dA_dt,dB_dt],dtype=float)
def rk4_step(t:float,y:np.ndarray,dt:float,
             params:Dict[str,float],d_uv:float)->np.ndarray:
    # One explicit RK4 integration step for y'=f(t,y).
    k1=biofilm_rhs(t,y,params,d_uv)
    k2=biofilm_rhs(t+dt/2,y+dt*k1/2,params,d_uv)
    k3=biofilm_rhs(t+dt/2,y+dt*k2/2,params,d_uv)
    k4=biofilm_rhs(t+dt,y+dt*k3,params,d_uv)
    y_next=y+(dt/6.0)*(k1+2*k2+2*k3+k4)
    return y_next
def integrate_biofilm(cond_id:str,params:Dict[str,float],
                      d_uv:float,dt:float,t_end:float,
                      A0:float=0.01,B0:float=0.01)->pd.DataFrame:
    # Integrates the biofilm system for a given condition.
    # Returns DataFrame with columns: t_h,A,B,condition_id,D_uv
    n_steps=int(np.floor(t_end/dt))+1
    t=np.linspace(0.0,t_end,n_steps)
    A=np.zeros_like(t)
    B=np.zeros_like(t)
    # initial conditions
    A[0]=A0
    B[0]=B0
    y=np.array([A0,B0],dtype=float)
    for i in range(1,n_steps):
        y=rk4_step(t[i-1],y,dt,params,d_uv)
        # enforce non-negativity
        y=np.maximum(y,0.0)
        A[i],B[i]=y
    df=pd.DataFrame({"t_h":t,"A":A,"B":B})
    df["condition_id"]=cond_id
    df["D_uv"]=d_uv
    return df
# MAIN
def main():
    # Pixel with highest f_illum
    row,col,f_illum=load_best_candidate(CANDIDATE_PATH)
    # Biological priors for the conditions of interest
    priors=load_priors(PRIORS_PATH,COND_IDS)
    # Effective UV dose from f_illum
    d_uv=uv_dose_from_fillum(f_illum,d_ref=D_REF)
    print(f"Effective UV dose D_uv={d_uv:.3f} (f_illum={f_illum:.3f})")
    # Integrate for each condition
    all_dfs=[]
    for cid in COND_IDS:
        print(f"Integrating biofilm for condition:{cid}")
        params=priors[cid]
        df_cond=integrate_biofilm(cond_id=cid,params=params,
                                  d_uv=d_uv,dt=DT_H,t_end=T_END_H,A0=0.01,B0=0.01)
        all_dfs.append(df_cond)
    # Combine and save
    out_df=pd.concat(all_dfs,ignore_index=True)
    out_path=BASE_DIR/"biofilm_timeseries.csv"
    out_df.to_csv(out_path,index=False)
    print(f"Saved:{out_path}")
    # (Optional) simple summary in console
    summary=out_df.groupby("condition_id")[["A","B"]].agg(["min","max"])
    print("Summary A/B by condition:")
    print(summary)
if __name__=="__main__":
    main()
```

2.11.6 biofilm_controllability_index.py

```
#!/usr/bin/env python3
# Computation of B_infty and C_bio from biofilm_timeseries.csv.
# Uses:
# - biofilm_timeseries.csv with columns: t_h,A,B,condition_id,D_uv
# Produces:
```

```
# - biofilm_steady_metrics.csv with: condition_id,D_uv,B_infty,A_infty,C_bio
import numpy as np
import pandas as pd
from pathlib import Path
# Base location (ADJUST)
BASE_DIR=Path("/home/a/Downloads/architecture/").resolve()
TS_PATH=BASE_DIR/"biofilm_timeseries.csv"
def compute_steady_values(df_cond:pd.DataFrame,frac_tail:float=0.1)->tuple[float,float]:
    # Returns (A_infty,B_infty) as the mean of A and B
    # over the last frac_tail of the time interval.
    t=df_cond["t_h"].to_numpy()
    A=df_cond["A"].to_numpy()
    B=df_cond["B"].to_numpy()
    t_max=t.max()
    t_cut=t_max*(1.0-frac_tail)
    mask=t>=t_cut
    A_inf=float(A[mask].mean())
    B_inf=float(B[mask].mean())
    return A_inf,B_inf
def main():
    df=pd.read_csv(TS_PATH)
    # ensure key columns
    for col in ["t_h","A","B","condition_id","D_uv"]:
        if col not in df.columns:
            raise ValueError(f"Missing column '{col}' in biofilm_timeseries.csv")
    # group by condition
    groups=df.groupby("condition_id")
    rows=[]
    for cond_id,g in groups:
        # assume D_uv constant per condition
        d_uv_vals=g["D_uv"].unique()
        if len(d_uv_vals) != 1:
            raise ValueError(f"Expected a single D_uv for condition_id={cond_id}")
        d_uv=float(d_uv_vals[0])
        A_inf,B_inf=compute_steady_values(g,frac_tail=0.1)
        rows.append({"condition_id":cond_id,"D_uv":d_uv,"A_infty":A_inf,"B_infty":B_inf})
    steady_df=pd.DataFrame(rows)
    # B_ref=worst case (maximum B_infty)
    B_ref=steady_df["B_infty"].max()
    steady_df["B_ref"]=B_ref
    steady_df["C_bio"]=1.0-steady_df["B_infty"]/B_ref
    out_path=BASE_DIR/"biofilm_steady_metrics.csv"
    steady_df.to_csv(out_path,index=False)
    print(f"Saved:{out_path}")
    print(steady_df)
if __name__=="__main__":
    main()
```

2.11.7 d_uv.py

```
#!/usr/bin/env python3
# Sweep of D_uv for OSD-554 conditions (priors) using the biofilm model.
# Uses:
# - P_bio_OSD554_priors_from_normalized.csv (priors)
# Produces:
# - biofilm_dose_response.csv with: condition_id,D_uv,B_infty,A_infty
import numpy as np
import pandas as pd
from pathlib import Path
from typing import Dict,List
# Base location (ADJUST)
BASE_DIR=Path("/home/a/Downloads/architecture/").resolve()
PRIORS_PATH=BASE_DIR/"P_bio_OSD554_priors_from_normalized.csv"
# Conditions to explore
COND_IDS:List[str]=["micro_SS","micro_LIS"]
# Range of D_uv (dimensionless)
D_UV_MIN=0.0
D_UV_MAX=2.0
N_D_UV=21   # number of points in the sweep
# Temporal integration parameters
T_END_H=240.0
DT_H=0.1
# Model parameters (must match the previous script)
C_B=1.0
K_S=0.5
K_A=0.05
def load_priors(path:Path,cond_ids:List[str])->Dict[str,Dict[str,float]]:
    df=pd.read_csv(path)
    required_cols={"condition_id","mu_max","k_d","Phi_0","beta_phi"}
    if not required_cols.issubset(df.columns):
        raise ValueError(f"Missing columns {required_cols} in priors.")
    priors={}
    for cid in cond_ids:
        row=df[df["condition_id"]==cid]
        if row.empty:
            raise ValueError(f"condition_id='{cid}' not found in priors.")
        r=row.iloc[0]
        priors[cid]={"mu_max":float(r["mu_max"]),"k_d":float(r["k_d"]),
            "Phi_0":float(r["Phi_0"]),"beta_phi":float(r["beta_phi"])
        }
    return priors
def biofilm_rhs(t:float,y:np.ndarray,params:Dict[str,float],d_uv:float)->np.ndarray:
```

```
    A,B=y
    mu_max=params["mu_max"]
    k_d=params["k_d"]
    Phi_0=params["Phi_0"]
    beta_phi=params["beta_phi"]
    Phi=Phi_0
    phi_rate=beta_phi*d_uv
    denom=K_S+max(A,0.0)
    dA_dt=K_A*Phi*C_B-(k_d+phi_rate)*A
    dB_dt=mu_max*(A/denom)*B-(k_d+phi_rate)*B
    return np.array([dA_dt,dB_dt],dtype=float)
def rk4_step(t:float,y:np.ndarray,dt:float,params:Dict[str,float],d_uv:float)->np.ndarray:
    k1=biofilm_rhs(t,y,params,d_uv)
    k2=biofilm_rhs(t+dt/2,y+dt*k1/2,params,d_uv)
    k3=biofilm_rhs(t+dt/2,y+dt*k2/2,params,d_uv)
    k4=biofilm_rhs(t+dt,y+dt*k3,params,d_uv)
    y_next=y+(dt/6.0)*(k1+2*k2+2*k3+k4)
    return np.maximum(y_next,0.0)
def integrate_steady(params:Dict[str,float],d_uv:float,t_end:float,dt:float,
                     A0:float=0.01,B0:float=0.01,frac_tail:float=0.1)->tuple[float,float]:
    n_steps=int(np.floor(t_end/dt))+1
    t=np.linspace(0.0,t_end,n_steps)
    A=np.zeros_like(t)
    B=np.zeros_like(t)
    y=np.array([A0,B0],dtype=float)
    A[0],B[0]=y
    for i in range(1,n_steps):
        y=rk4_step(t[i-1],y,dt,params,d_uv)
        A[i],B[i]=y
    # asymptotic values as the mean over the last frac_tail
    t_max=t.max()
    t_cut=t_max*(1.0-frac_tail)
    mask=t>=t_cut
    A_inf=float(A[mask].mean())
    B_inf=float(B[mask].mean())
    return A_inf,B_inf
def main():
    priors=load_priors(PRIORS_PATH,COND_IDS)
    d_uv_values=np.linspace(D_UV_MIN,D_UV_MAX,N_D_UV)
    rows=[]
    for cid in COND_IDS:
        params=priors[cid]
        for d_uv in d_uv_values:
            A_inf,B_inf=integrate_steady(params,d_uv,T_END_H,DT_H)
            rows.append({"condition_id":cid,"D_uv":d_uv,"A_infty":A_inf,"B_infty":B_inf})
            print(f"{cid},D_uv={d_uv:.2f}->B_infty={B_inf:.4f}")
    dose_df=pd.DataFrame(rows)
    out_path=BASE_DIR/"biofilm_dose_response.csv"
    dose_df.to_csv(out_path,index=False)
    print(f"Saved:{out_path}")
if __name__=="__main__":
    main()
```

3 Results

3.1 Steady–State Biofilm Behavior and Controllability at the Selected Ridge Site

For the polar candidate with the highest illumination fraction, the horizon/solar–track pipeline yields an effective UV dose parameter

$$D_{\mathrm{uv}} \approx 0.756.$$

Using this dose, the reduced biofilm model was integrated to steady state for the two microgravity/material conditions derived from OSD–554 priors:

- **microgravity, SS–like inner liner (`micro_SS`):**

$$A_\infty \approx 1.94 \times 10^{-1}, \qquad B_\infty \approx 5.23 \times 10^{-32}.$$

- **microgravity, LIS–like inner liner (`micro_LIS`):**

$$A_\infty \approx 3.09 \times 10^{-2}, \qquad B_\infty \approx 5.03 \times 10^{-97}.$$

The reference biomass is defined as the largest steady-state biomass among the scenarios considered,

$$B_{\mathrm{ref}} = \max(B_\infty) = B_\infty(\texttt{micro_SS}) \approx 5.23 \times 10^{-32},$$

and the biofilm controllability index is

$$\mathcal{C}_{\mathrm{bio}} = 1 - \frac{B_\infty}{B_{\mathrm{ref}}}.$$

By construction this yields

$$\mathcal{C}_{\mathrm{bio}}(\texttt{micro_SS}) = 0, \qquad \mathcal{C}_{\mathrm{bio}}(\texttt{micro_LIS}) \approx 1.$$

In absolute terms both steady-state biomasses are numerically negligible at this dose level, but the ratio

$$\frac{B_\infty(\texttt{micro_LIS})}{B_\infty(\texttt{micro_SS})} \sim 10^{-65}$$

indicates that, within the model's internal scaling, LIS–like treatments are driven even deeper into the radiatively dominated suppression regime than SS–like liners at the same site. Thus the `micro_SS` case defines the "least controlled" baseline at vanishing biomass, and the `micro_LIS` case lies effectively at the upper end of the $\mathcal{C}_{\mathrm{bio}} \in [0, 1]$ scale. This qualitative ordering—SS-like liners as high-attachment, high-biomass baselines and LIS-like liners as low-attachment, low-biomass analogues—is consistent with ISS-relevant experiments comparing stainless steel and LIS-type surfaces under microgravity and with broader literature on low-adhesion surfaces for biofilm mitigation [10–12, 37, 40].

3.2 Dose–Response Curves $B_\infty(D_{\mathrm{uv}})$

To decouple site-specific illumination from intrinsic material response, the same priors were used to generate dose–response curves over

$$D_{\mathrm{uv}} \in [0, 2].$$

For each condition and each value of D_{uv}, the system was integrated to steady state and the asymptotic biomass $B_\infty(D_{\mathrm{uv}})$ was estimated as the time-average over the last 10% of the integration window.

3.2.1 SS–like Condition (`micro_SS`)

The model exhibits pronounced threshold behavior:

- At $D_{\mathrm{uv}} = 0$, the system converges to a very large biomass (order 10^{10}–10^{11}, truncated in the exported table).
- For $D_{\mathrm{uv}} = 0.1$–0.2, biomass remains extremely high; at $D_{\mathrm{uv}} = 0.3$ the system passes through a sharp maximum:

  $$B_\infty(\texttt{micro_SS}, D_{\mathrm{uv}} = 0.3) \approx 1.74 \times 10^{7}.$$

- Between $D_{\mathrm{uv}} = 0.3$ and 0.4, a damage-driven threshold is crossed. At $D_{\mathrm{uv}} = 0.4$:

  $$B_\infty(\texttt{micro_SS}, 0.4) \approx 2.1 \times 10^{-3},$$

  a drop of $\sim 10^{10}$–10^{11} relative to the peak.
- For $D_{\mathrm{uv}} \geq 0.5$, biomass decays monotonically by many orders of magnitude:

  $$B_\infty(\texttt{micro_SS}, 0.5) \approx 3.1 \times 10^{-12},$$
  $$B_\infty(\texttt{micro_SS}, 0.7) \approx 6.1 \times 10^{-28},$$
  $$B_\infty(\texttt{micro_SS}, 1.0) \approx 5.8 \times 10^{-49},$$
  $$B_\infty(\texttt{micro_SS}, 2.0) \approx 1.7 \times 10^{-111}.$$

Thus an SS–like liner exhibits a broad high-biomass regime for $D_{\mathrm{uv}} \lesssim 0.3$, followed by abrupt collapse beyond $D_{\mathrm{uv}} \approx 0.4$. This threshold-like response is qualitatively similar to UV dose–response curves reported for *P. aeruginosa* biofilms under UVC and UV-LED exposure, where biofilm production or survival can increase or remain stable up to a strain-specific threshold dose and then decline sharply at higher doses [35, 37, 38].

3.2.2 LIS–like Condition (`micro_LIS`)

Unlike SS, LIS shows no intermediate maximum:

- At $D_{\mathrm{uv}} = 0$:
$$B_\infty(\texttt{micro_LIS}, 0) \approx 3.73 \times 10^{10}.$$
- At $D_{\mathrm{uv}} = 0.1$, biomass collapses immediately:
$$B_\infty(\texttt{micro_LIS}, 0.1) \approx 9.3 \times 10^{-12}.$$
- For $D_{\mathrm{uv}} \geq 0.2$, biomass is effectively zero:
$$B_\infty(\texttt{micro_LIS}, 0.2) \approx 5.1 \times 10^{-27},$$
$$B_\infty(\texttt{micro_LIS}, 0.5) \approx 6.9 \times 10^{-66},$$
$$B_\infty(\texttt{micro_LIS}, 1.0) \approx 2.5 \times 10^{-126},$$
$$B_\infty(\texttt{micro_LIS}, 2.0) \approx 1.1 \times 10^{-244}.$$

Once any nonzero UV damage term is present, LIS is driven directly into a suppressed regime. This is consistent with experimental findings that low-adhesion or LIS-like surfaces exhibit reduced initial biofilm formation and increased UV inactivation efficiency relative to stainless steel and other conventional materials [11, 34, 37].

3.3 Implications for the Selected Polar Microhabitat Design

The selected ridge site, with $f_{\mathrm{illum}} \approx 0.76$ and corresponding

$$D_{\mathrm{uv}} \approx 0.756,$$

lies deeply inside the high-damage, low-biomass regime for both material classes:

- For `micro_SS`, the range $D_{\mathrm{uv}} = 0.7$–0.8 yields biomass of order 10^{-28}–10^{-35}, consistent with the site value $B_\infty \approx 5.2 \times 10^{-32}$.
- For `micro_LIS`, the same range gives $B_\infty \sim 10^{-90}$–10^{-100}, consistent with $B_\infty \approx 5.0 \times 10^{-97}$.

Thus even an SS–like inner surface hosts only residual steady-state biomass, while an LIS–like analog further suppresses biomass by $\sim$ 60–70 additional orders of magnitude. SS shows a sharp threshold near $D_{\mathrm{uv}} \sim 0.3$–0.4; if shading reduced the dose below this range, SS liners would re-enter a high-biomass regime, whereas LIS would remain suppressed for $D_{\mathrm{uv}} \gtrsim 0.1$. In architectural terms, the combined illumination–biofilm model implies: (i) a low UV threshold $D_{\mathrm{uv}} \sim 0.3$–0.4 above which SS transitions to negligible biofilms, and (ii) a broader, more robust suppression domain for LIS, requiring only $D_{\mathrm{uv}} \gtrsim 0.1$ to achieve effectively sterilized steady states, in qualitative agreement with UV-based biofilm control strategies on engineered surfaces [35, 37, 40].

4 Discussion

The integrated results highlight several robust structural features of the design space, even under the deliberately simplified physics and biology adopted here. On the physical side, the polar horizon analysis confirms that only a small subset of ridge and crater-rim locations achieve high fractional illumination over a precession cycle, consistent with prior studies based on LOLA and long-term solar-track simulations [1, 2]. Within this subset, the combination of shell geometry and Diviner-derived thermophysical properties controls the internal climate: small increases in shell thickness or modest changes in regolith thermal inertia can shift minimum internal temperatures by tens of kelvin, while leaving the overall sintering energy budget within feasible bounds for solar-driven processes [3, 5, 6].

From the biological perspective, the most striking outcome is the strong separation in steady-state biofilm biomass between SS-like and LIS-like inner liners at a fixed site-level UV dose. When the reduced model is evaluated with parameter ensembles derived from ISS biofilm experiments, LIS-like analogues occupy the high-controllability end of the $\mathcal{C}_{\mathrm{bio}} \in [0, 1]$ scale, whereas SS-like liners cluster near low controllability. This ordering mirrors experimental observations that liquid-infused and other low-adhesion surfaces suppress biofilm formation and corrosion relative to conventional stainless-steel coupons in BAC and related studies, even under microgravity [10–12]. In absolute terms, the steady-state biomasses predicted by the reduced model at the most strongly illuminated ridge sites are extremely low, but the relative differences across materials are large, suggesting substantial leverage in material selection and liner treatment.

These patterns should not be interpreted as quantitative predictions of biofilm loads in any specific mission scenario. The thermal and illumination models neglect many second-order effects, including 3D conduction through penetrations and interfaces, complex radiative coupling to internal hardware, dust deposition on external surfaces, and potential changes in regolith properties due to repeated sintering cycles. On the biological side, the reduced model collapses a high-dimensional space of microbial consortia, EPS rheology, nutrient availability, flow regimes, and multi-species interactions into a two-variable ODE system. Moreover, the parametrization uses ISS-derived priors from a limited set of organisms and materials, and extrapolates them to polar microhabitats with very different environmental forcing.

Within these limitations, the framework still occupies a useful niche in the broader landscape of mission and space microbiology. It demonstrates that open planetary datasets and open microbiology experiments can be integrated into a single, legible pipeline that runs on commodity hardware and exposes clear design knobs: shell radius, height, and thickness; polar site selection; liner material class; and acceptable ranges of internal temperature and biofilm controllability. The model can thus serve as a low-cost, low-latency front-end that informs which regions of the design space merit more detailed, high-fidelity simulations or experimental campaigns.

The results also suggest several directions for refinement. On the physical side, more realistic 3D thermal models could incorporate detailed internal layouts and radiative coupling, while illumination modeling could be extended to include time-varying dust and surface roughness at sub-grid scales. On the biological side, the reduced model could be replaced or supplemented by higher-dimensional dynamical systems or data-driven surrogates trained on emerging ISS biofilm datasets and terrestrial analog experiments, including community-level models that capture interactions among bacteria, fungi, and archaea. Ultimately, the most valuable extension may be a closed loop in which architectural design variables are directly coupled to experimental campaigns on biofilm formation under controlled UV, temperature, and material conditions, closing the gap between theoretical controllability indices and operational risk metrics.

5 Conclusions

This work introduces and evaluates a deliberately constrained yet extensible framework for thinking about sintered-regolith microhabitats at the lunar poles as coupled thermo–biofilm systems. By parameterizing spherical-cap shells, linking them to polar illumination and Diviner-derived thermophysical products, and embedding a reduced biofilm model calibrated against ISS-derived priors, we obtain a compact set of dimensionless indices that summarize illumination, sintering energy, internal climate, and biofilm controllability in an architecturally legible form [1–3, 11].

When evaluated at representative high-illumination ridge sites, the model suggests that thin, sintered shells with LIS-like inner liners can simultaneously maintain internal temperatures within a habitable envelope and strongly suppress steady-state biofilm biomass relative to SS-like liners at the same UV dose. Although the absolute values of the predicted biomasses are not to be trusted quantitatively, the relative ordering across materials aligns with experimental observations from BAC and Space Biofilms investigations, where liquid-infused and other low-adhesion surfaces exhibit reduced biofilm coverage and corrosion under microgravity [10–12].

The main contribution is therefore methodological rather than numerical. The framework demonstrates that (i) open planetary datasets (LOLA, LROC, Diviner) and open microbiology datasets (ISS metagenomes, spaceflight biofilm experiments) can be co-embedded into a single pipeline; (ii) the resulting models are lightweight enough to be executed on standard CPUs, making them accessible to architectural and systems-design communities without dedicated HPC; and (iii) the outputs can be expressed as scalar indices and trade-off maps that integrate naturally into early-stage design workflows.

Finally, the results underscore the value of open science for habitats that must coexist with microbial life. As lunar and other off-world infrastructures are designed, updated measurements of polar illumination, regolith properties, and in-habitat microbiomes will become available. Incorporating these into transparent, reproducible pipelines—such as the one explored here—will be essential for moving from qualitative statements about "biofilm risk" to quantitative, design-coupled metrics that respect both the physical and biological constraints of long-term human presence beyond Earth.

6 Declarations

Availability of data and material. All data, models, and computational materials supporting the findings of this study are available in the document. Publicly accessible datasets obtained from the NASA Open Science Data Repository (OSDR) were used in accordance with their standard usage policies.

Competing interests. The author declares that there are no competing interests.

Funding. This work was fully funded by Grupo Alianza Empresarial. The funder had no role in the study design, data analysis, interpretation of results, or manuscript preparation.

Authors' contributions. The author solely conceived the study, developed the methodology, performed all analyses, created all figures and models, and wrote the manuscript.

Acknowledgements. The author acknowledges the NASA Open Science Data Repository (OSDR) for providing access to publicly available datasets used in this study. NASA had no involvement in the design, analysis, interpretation, or conclusions of this work.

References

[1] E. Mazarico, G. A. Neumann, D. E. Smith, M. T. Zuber, and M. H. Torrence. Illumination conditions of the lunar polar regions using lola topography. *Icarus*, 211(2):1066–1081, 2011. ISSN 0019-1035. doi: 10.1016/j.icarus.2010.10.030.

[2] P. Gläser, J. Oberst, G. A. Neumann, E. Mazarico, E. J. Speyerer, and M. S. Robinson. Illumination conditions at the lunar poles: Implications for future exploration. *Planetary and Space Science*, 162:170–178, 2018. ISSN 0032-0633. doi: 10.1016/j.pss.2017.07.006.

[3] P. O. Hayne, M. A. Bandfield, J. L. Siegler, A. R. Vasavada, R. R. Ghent, J. P. Williams, B. T. Greenhagen, O. Aharonson, C. M. Elder, P. G. Lucey, and D. A. Paige. Global regolith thermophysical properties of the moon from the diviner lunar radiometer experiment. *Journal of Geophysical Research: Planets*, 122(12):2371–2400, 2017. doi: 10.1002/2017JE005387.

[4] K. Baxter, E. Sas, K. Clark, M. Walsh, N. Pradeep, A. Batool, C. Naney, M. Vargas, N. Kennerdale, K. Das, Z. Shi, A. Kelam, V. Verma, M. Simões, D. Neefs, V. Ravichandran, M. Tirumalai, Barbero B., G. Macori, E. Gonzalez, B. Sikes, F. Karouia, and N. Brereton. Biofilms: From the cradle of life to life support. *npj Biofilms Microbiomes, in press*, 2025.

[5] A. Meurisse, A. Makaya, C. Willsch, and M. Sperl. Solar 3D printing of lunar regolith. *Acta Astronautica*, 152:800–810, 2018. ISSN 0094-5765. doi: 10.1016/j.actaastro.2018.06.063.

[6] R. Zhang, T. Wang, and G. Wang. Review of lunar regolith forming technologies for in-situ manufacturing/construction on the lunar surface. *Additive Manufacturing Frontiers*, 4(3): 200238, 2025. ISSN 2950-4317. doi: 10.1016/j.amf.2025.200238.

[7] A. A. Voorhies, C. Mark Ott, S. Mehta, D. L. Pierson, B. E. Crucian, A. Feiveson, C. M. Oubre, M. Torralba, K. Moncera, Y. Zhang, E. Zurek, and H. A. Lorenzi. Study of the impact of long-duration space missions at the International Space Station on the astronaut microbiome. *Scientific Reports*, 9(1):9911, 2019. doi: 10.1038/s41598-019-46303-8.

[8] R. A. Salido, H. N. Zhao, D. McDonald, H. Mannochio-Russo, S. Zuffa, R. E. Oles, A. T. Aron, Y. El Abiead, S. Farmer, A. González, C. Martino, I. Mohanty, C. W. Parker, L. Patel, P. W. Portal Gomes, R. Schmid, T. Schwartz, J. Zhu, M. R. Barratt, K. H. Rubins, H. Chu, F. Karouia, K. Venkateswaran, P. C. Dorrestein, and R. Knight. The international space station has a unique and extreme microbial and chemical environment driven by use patterns. *Cell*, 188(7):2022–2041.e23, 2025. ISSN 0092-8674. doi: 10.1016/j.cell.2025.01.039.

[9] K. Landry, J. Morey, B. Bharat, N. Haney, and S. Panesar. Biofilms—Impacts on human health and its relevance to space travel. *Microorganisms*, 8(7), 2020. ISSN 2076-2607. doi: 10.3390/microorganisms8070998.

[10] D. Goeres, Y. Velez, B. Kjellerup, and R. McLean. Biofilm and human spaceflight. *Biofilm*, 6:100150, 2023. ISSN 2590-2075. doi: 10.1016/j.bioflm.2023.100150.

[11] OSD-554. Characterization of Biofilm Formation, Growth, and Gene Expression on Different Materials and Environmental Conditions in Microgravity (Gene expression of Pseudomonas aeruginosa biofilms). NASA OSDR, 2025. URL `https://osdr.nasa.gov/bio/repo/data/studies/OSD-554`.

[12] Polymicrobial Biofilm Growth and Control during Spaceflight (Bacterial Adhesion and Corrosion). NASA, 2023. URL `https://science.nasa.gov/biological-physical/investigations/bacterial-adhesion/`.

[13] S. Timoshenko and S. Woinowsky-Krieger. *Theory of plates and shells*. McGraw–Hill, 1964.

[14] V. Terzi and T. Makarios. Finite Element In-Depth Verification: Base Displacements of a Spherical Dome Loaded by Edge Forces and Moments. *Modelling*, 5(1):37–54, 2024. ISSN 2673-3951. doi: 10.3390/modelling5010003.

[15] S. Wood. An analytic model for the thermal conductivity of planetary regolith: Uncemented, non-spherical particulates. In *44th Lunar and Planetary Science Conference*, number 3077, 2013.

[16] Gläser, P. and Gläser, D. Modeling near-surface temperatures of airless bodies with application to the moon. *Astronomy & Astrophysics*, 627:A129, 2019. doi: 10.1051/0004-6361/201935514.

[17] E. Mazarico, M. Barker, and J. Nicholas. Advanced illumination modeling for data analysis and calibration. Application to the Moon. *Advances in Space Research*, 62(11):3214–3228, 2018. ISSN 0273-1177. doi: 10.1016/j.asr.2018.08.022.

[18] Y. Zhang, B. Liu, K. Di, S. Liu, Z. Yue, S. Han, J. Wang, W. Wan, and B. Xie. Analysis of Illumination Conditions in the Lunar South Polar Region Using Multi-Temporal High-Resolution Orbital Images. *Remote Sensing*, 15(24), 2023. ISSN 2072-4292. doi: 10.3390/rs15245691.

[19] X. Tong, Q. Huang, S. Liu, H. Xie, H. Chen, Y. Wang, X. Xu, C. Wang, and Y. Jin. A high-precision horizon-based illumination modeling method for the lunar surface using pyramidal LOLA data. *Icarus*, 390:115302, 2023. ISSN 0019-1035. doi: 10.1016/j.icarus.2022.115302.

[20] M. Barker, E. Mazarico, G. Neumann, D. Smith, M. Zuber, and J. Head. Improved lola elevation maps for south pole landing sites: Error estimates and their impact on illumination conditions. *Planetary and Space Science*, 203:105119, 2021. ISSN 0032-0633. doi: 10.1016/j.pss.2020.105119.

[21] A. Ryan, D. Pino, M. Bernacki, and M. Delbo. Full-field modeling of heat transfer in asteroid regolith: 1. radiative thermal conductivity of polydisperse particulates. *Journal of Geophysical Research: Planets*, 125(2), 2020. doi: 10.1029/2019JE006100.

[22] A New View of the Lunar South Pole from LOLA. NASA's Planetary Geology, Geophysics and Geochemistry Laboratory, 2020. URL `https://pgda.gsfc.nasa.gov/products/90`.

[23] M. Barker, E. Mazarico, G. Neumann, D. Smith, M. Zuber, J. Head, and X. Sun. A New View of the Lunar South Pole from the Lunar Orbiter Laser Altimeter (LOLA). *The Planetary Science Journal*, 4(9):183, sep 2023. doi: 10.3847/PSJ/acf3e1.

[24] K. Ogawa, N. Sakatani, Y. Iijima, M. Arakawa, R. Honda, and S. Tanaka. Thermal conductivity model for powdered materials under vacuum based on experimental studies. *AIP Advances*, 7(1):015310, 01 2017. ISSN 2158-3226. doi: 10.1063/1.4975153.

[25] S. Lim, J. Bowen, G. Degli-Alessandrini, M. Anand, A. Cowley, and V. Levin Prabhu. Investigating the microwave heating behaviour of lunar soil simulant jsc-1a at different input powers. *Scientific Reports*, 11(1):2133, 2021. doi: 10.1038/s41598-021-81691-w.

[26] A. Gatto, S. Defanti, E. Bassoli, A. Mattioni, U. Martini, and G. Incerti. Preliminary study on localized microwave sintering of lunar regolith. *Acta Astronautica*, 218:126–136, 2024. ISSN 0094-5765. doi: 10.1016/j.actaastro.2024.02.026.

[27] E. Ethridge and W. Kaukler. Microwave Extraction of Water from Lunar Regolith Simulant. *AIP Conference Proceedings*, 880(1):830–837, 01 2007. ISSN 0094-243X. doi: 10.1063/1.2437523.

[28] S Tsubaki, S Fujii, M Yamamoto, H Kanamori, T Hoshino, S Hosoda, and Y. Wada. Self-sufficient production of lunar regolith gravels on the moon by ultrarapid microwave sintering. *ACS Omega*, 9(20), 2024. doi: 10.1021/acsomega.4c02702.

[29] S. Tao, B. Vinh, N. Ceshkovsky, and J. Chang. MOONPads: Microwave-Sintering Operations Of Nanophase-Iron Pads,howpublished=NASA Human Lander Challenge Technical Paper, 2024. URL `https://hulc.nianet.org/wp-content/uploads/2024-HuLC-Chart-Deck-University-of-Calififormia-San-Diego-MOON-Pads.pdf`.

[30] H. Eberl, E. Morgenroth, D. Noguera, C. Picioreanu, B. Rittmann, M. van Loosdrecht, and O. Wanner. *Mathematical Modelling of Biofilms*. IWA Publishing, 07 2006. ISBN 9781780402482. doi: 10.2166/9781780402482.

[31] F. Abbas. *Mathematical Contributions to One-Dimensional Biofilm Models*. PhD thesis, University of Guelph, 2011.

[32] S. Skoneczny and M. Cioch-Skoneczny. Approximate Models of Microbiological Processes in a Biofilm Formed on Fine Spherical Particles. *Processes*, 10(1), 2022. ISSN 2227-9717. doi: 10.3390/pr10010048.

[33] H. Eberl, C. Picioreanu, M. van Loosdrecht, J.J. Heijnen, and J. Kreft. Mathematical Modelling of Biofilm Structures. *Antonie van Leeuwenhoek*, 81:245–256, 2002. doi: 10.1023/A:1020527020464.

[34] T. Mullin, S. MacIsaac, A. Stoddart, and G. Gagnon. Impact of Material Properties for Improved Pseudomonas aeruginosa Biofilm Inactivation by UV-LEDs. *Scientific Reports*, 15, 2025. doi: 10.1038/s41598-025-15192-5.

[35] S. Gora, K. Rauch, C. Ontiveros, A. Stoddart, and G. Gagnon. Inactivation of biofilm-bound Pseudomonas aeruginosa bacteria using UVC light emitting diodes (UVC LEDs). *Water Research*, 151:193–202, 2019. ISSN 0043-1354. doi: 10.1016/j.watres.2018.12.021.

[36] B. Ma, S. Seyedi, E. Wells, D. McCarthy, N. Crosbie, and K. Linden. Inactivation of biofilm-bound bacterial cells using irradiation across UVC wavelengths. *Water Research*, 217:118379, 2022. ISSN 0043-1354. doi: 10.1016/j.watres.2022.118379.

[37] S. Gora, B. Ma, M. Lanzarini-Lopes, H. Torkzadeh, Z. Zhao, C. Ley Matthews, P. Westerhoff, K. Linden, B. Barbeau, R. Simons, G. Gagnon, P. Di Falco, and M. Mohsin. Control of biofilms with uv light: a critical review of mechanisms and applications. *Environmental Science: Water Research & Technology*, 10(12):2205–2236, 2024. doi: 10.1039/D4EW00506F.

[38] M. Said, I. Daly, H. Nasr, and A. Hassen. Monitoring of biofilm production by pseudomonas aeruginosa after uvc irradiation: Dose/response modelling. *Annals of Microbiology*, 63, 2013. doi: 10.1007/s13213-012-0487-7.

[39] C. Jones, S. Valdeig, R. Sova, and C. Weiss. Inside-out Ultraviolet-C Sterilization of Pseudomonas aeruginosa Biofilm In Vitro. *Photochemistry and Photobiology*, 92(6):835–841, 2016. doi: 10.1111/php.12637.

[40] U. Waqas, A. Farhan, A. Haider, U. Qumar, and A. Raza. Advancements in biofilm formation and control in potable water distribution systems: A comprehensive review and analysis of chloramine decay in water systems. *Journal of Environmental Chemical Engineering*, 11(6):111377, 2023. ISSN 2213-3437. doi: 10.1016/j.jece.2023.111377.

[41] References, 2025. URL `https://www.miguelangelvargascruz.com/en/references`.